wt-orchestrate

How It Works --- The Complete Orchestration Pipeline

March 2026

wt-tools // autonomous orchestration framework

>_ built with Claude Code agents

Contents

1 25 Years Later . . . 6

2 Why Is This Needed? . . . 7
- 2.1 The Problem . . . 7
- 2.2 The Solution: Autonomous Orchestration . . . 7
- 2.3 Who Is This For? . . . 8
- 2.4 How Is This Different from CI/CD? . . . 8

3 Development History . . . 9

4 The wt-tools Ecosystem . . . 10
- 4.1 Overview . . . 10
- 4.2 Core Tools . . . 10
  - 4.2.1 Worktree Lifecycle . . . 10
  - 4.2.2 Agent Execution . . . 10
  - 4.2.3 Orchestration and Supervision . . . 11
  - 4.2.4 Memory and Context . . . 11
  - 4.2.5 Project Management . . . 11
  - 4.2.6 Team Synchronization . . . 12
  - 4.2.7 OpenSpec and Quality . . . 12
- 4.3 Project Templates . . . 12
  - 4.3.1 wt-project-base . . . 12
  - 4.3.2 wt-project-web (Next.js) . . . 12
  - 4.3.3 Other Directions . . . 13
- 4.4 Ecosystem Map . . . 13

5 Overview . . . 14
- 5.1 What Is Orchestration? . . . 14
- 5.2 The 5-Layer Model . . . 14
- 5.3 Modular Architecture . . . 15
- 5.4 State Files . . . 15
- 5.5 A Typical Run . . . 17

6 Input and Configuration . . . 18
- 6.1 Input Modes . . . 18
  - 6.1.1 1. Specification Mode (`--spec`) . . . 18
  - 6.1.2 2. Brief Mode (`--brief`) . . . 19
  - 6.1.3 3. Auto-detect . . . 19
- 6.2 Configuration . . . 19
  - 6.2.1 orchestration.yaml . . . 19
  - 6.2.2 Document Directives . . . 20
- 6.3 Directive Reference . . . 20

7 Digest and Triage . . . 22
- 7.1 What Is a Digest? . . . 22
- 7.2 Digest Generation Steps . . . 22
  - 7.2.1 1. Spec Scanning (`scan_spec_directory`) . . . 22
  - 7.2.2 2. Prompt Assembly (`build_digest_prompt`) . . . 22
  - 7.2.3 3. API Call and Parsing . . . 22
  - 7.2.4 4. ID Stabilization (`stabilize_ids`) . . . 24
- 7.3 Digest Output Files . . . 24

7.4 Freshness Check 24
7.5 Ambiguity Triage 24
7.5.1 Triage Generation 25
7.5.2 Human Decision 25
7.5.3 Automatic Triage Merging 25
7.6 Requirement Coverage 25

8 OpenSpec --- The Spec-Driven Workflow 26
8.1 What Is OpenSpec? 26
8.2 The Artifacts 26
8.2.1 Proposal 26
8.2.2 Design 26
8.2.3 Specs 26
8.2.4 Tasks 26
8.3 Integration with Orchestration 27
8.3.1 Dispatch Phase 27
8.3.2 Ralph Loop Integration 27
8.3.3 FF→Apply Chaining 27
8.4 Interactive vs. Orchestrated Usage 27
8.5 Traceability 27
8.6 Limitations 28

9 Planning 29
9.1 The Purpose of Decomposition 29
9.2 Planning Methods 29
9.2.1 API Mode (default) 29
9.2.2 Agent Mode 29
9.3 The Plan Structure 31
9.3.1 Field Descriptions 31
9.4 Topological Sort (DAG) 32
9.4.1 Circular Dependency Detection 32
9.5 Test Infrastructure Discovery 32
9.6 Spec Summarization for Large Documents 32
9.7 Plan Validation 33

10 Execution 34
10.1 Dispatch: The Change Begins 34
10.1.11. Worktree Creation 34
10.1.22. Bootstrap 35
10.1.33. Synchronization with Main Branch 35
10.1.44. Ralph Loop Start 35
10.2 The Ralph Loop 35
10.2.1Iteration Cycle 35
10.2.2loop-state.json 35
10.2.3Completion Conditions 36
10.2.4Context Pruning 36
10.3 Model Routing 36
10.3.1Complexity-Based Routing 36
10.4 Parallel Execution 37
10.5 Pause and Resume 37
10.6 Token Tracking 37

11 The Sentinel 38
11.1 Why Does the Supervisor Need a Supervisor? 38
11.2 Bash Sentinel 38
11.2.1Crash Recovery with Exponential Backoff 38
11.2.2Liveness Detection 38
11.2.3State Recovery 39
11.2.4Exit Classification 39
11.3 Agent Sentinel 39
11.3.1Tiered Intervention 39

11.3.2 Checkpoint Handling 40
11.3.3 Autonomy Rules 40
11.4 Sentinel Events 40

12 Monitor and Watchdog 41
12.1 The Monitor Loop 41
12.1.1 The Poll Cycle 41
12.1.2 poll_change() 41
12.1.3 Active Time Tracking 41
12.2 The Watchdog System 43
12.2.1 Detection Mechanisms 43
12.2.1.1 1. Timeout Detection 43
12.2.1.2 2. Action Hash Loop Detection 43
12.2.1.3 3. Artifact Creation Grace Period 43
12.2.2 Escalation Levels 43
12.2.3 Recovery Detection 45
12.3 Token Safety Nets 45
12.3.1 Soft Limit (`token_budget`) 45
12.3.2 Hard Limit (`token_hard_limit`) 45
12.4 Time Limit 46
12.5 Memory and Auditing 46

13 Quality Gates 47
13.1 Overview 47
13.2 Test Gate 47
13.2.1 How It Works 47
13.2.2 Parameters 47
13.2.3 Output Handling 48
13.3 Review Gate 48
13.3.1 How It Works 48
13.3.2 Requirement-Aware Review 48
13.3.3 Configuration 48
13.4 Verify Gate 49
13.4.1 How It Works 49
13.4.2 Retry Logic 49
13.5 Smoke Gate 49
13.5.1 How It Works 49
13.5.2 Blocking vs Non-blocking 49
13.5.3 Smoke Fix Cycle 50
13.5.4 Health Check 50
13.6 E2E Gate 50
13.6.1 Two Modes 50
13.7 Gate Order Summary 51
13.8 Hooks 51

14 Merge and Delivery 52
14.1 Merge Queue 52
14.2 Merge Policy 52
14.2.1 eager 52
14.2.2 checkpoint (default) 52
14.2.3 manual 52
14.3 The Merge Process 54
14.3.1 Case 1: Branch No Longer Exists 54
14.3.2 Case 2: Branch Already Merged 54
14.3.3 Case 3: Normal Merge (the common case) 54
14.4 3-Layer Conflict Resolution 54
14.4.1 Layer 1: Generated Files (automatic) 54
14.4.2 Layer 2: LLM Merge (`--llm-resolve`) 54
14.4.3 Layer 3: Human Intervention 55
14.5 Post-Merge Pipeline 55
14.5.1 1. Running Worktree Synchronization 55

14.5.22. Base Build Check 55
14.5.33. Post-Merge Command 55
14.5.44. Change Archival 55
14.5.55. Post-Merge Hook 55
14.5.66. Coverage Update 56
14.6 Checkpoint and Approval 56
14.6.1 Checkpoint Activation 56
14.6.2 Approval Process 56
14.6.3 Auto-approve (for E2E runs) 56
14.7 Merge Retry Intelligence 56

15 Lessons Learned and Challenges 57
15.1 Solved Problems 57
15.1.1 Token Budget Calibration 57
15.1.2 Watchdog Spam 57
15.1.3 Sentinel Filename Mismatch 57
15.1.4 Misleading Token Tracking 57
15.1.5 Large Change Reliability 58
15.1.6 Jest + Playwright Collision 58
15.2 Known Limitations 58
15.2.1 Mock-Based Tests Hide Runtime Errors 58
15.2.2 Merge Conflicts on Shared Resources 58
15.2.3 Context Loss After Merge 58
15.2.4 Memory Doesn't Guarantee Quality 58
15.3 What's Still Ahead 59
15.3.1 Cascade Failure Logic 59
15.3.2 Trend-Based Token Budget 59
15.3.3 Agent Scoring for Dispatch 59
15.3.4 Circuit Breaker for API Calls 59

16 Replan and Coverage 60
16.1 Auto-Replan 60
16.1.1 Activation 60
16.1.2 Process 60
16.1.3 Retry Logic 60
16.1.4 All-Done Detection 60
16.2 Manual Replan 61
16.3 Requirement Coverage 61
16.3.1 Coverage Report 61
16.3.2 Coverage Tracking Lifecycle 62
16.3.3 Coverage Gap Detection 62
16.3.4 Cross-Cutting Requirements 62
16.4 Final Coverage Check 63
16.5 Summary Reports 63
16.5.1 HTML Report 63
16.5.2 Summary Email 63
16.5.3 orchestration-summary.md 63

17 Reference 64
17.1 CLI Reference 64
17.1.1 wt-orchestrate 64
17.1.1.1 Global Options 64
17.1.1.2 Commands 64
17.1.2 wt-new / wt-merge / wt-loop 65
17.2 Change State Machine 65
17.2.1 States 66
17.2.2 Key State Transitions 66
17.3 Directive Table 66
17.3.1 Execution 66
17.3.2 Models 67
17.3.3 Testing 67

17.3.4 Review and Verify . . . 67
17.3.5 Token Control . . . 68
17.3.6 Planning . . . 68
17.3.7 Watchdog . . . 68
17.3.8 Hooks . . . 68
17.3.9 Event Log . . . 69
17.4 File Structure Reference . . . 69
17.4.1 Project-Level Files . . . 69
17.4.2 Worktree-Level Files . . . 69
17.4.3 Event Types . . . 70

Chapter 1

25 Years Later

In the early 2000s, I worked in Hungary as part of a Nokia subcontractor team. We built SMS gateways and similar telecommunications infrastructure — the big infrastructure projects of that era, where reliability wasn't optional but a baseline requirement. Millions of users' messages flowed through our servers, and every minute of downtime had a measurable cost. That's where I learned what *systems-level thinking* means: it's not enough for a single component to work — the entire pipeline needs to work, end-to-end, day and night, without human intervention. When something stopped, a human didn't go look at it first — the system tried to fix itself.

Twenty-five years later, in a completely different technological context, the same pattern resurfaced. `wt-orchestrate` solves the same problem: how to run *autonomous, multi-step pipelines* where the system handles errors on its own, escalates on its own, and only asks a human when it truly has to. Only here it's not SMS messages flowing through the pipeline, but *software development tasks*. The "agent" isn't a hardware node but a Claude Code session in a git worktree. The "watchdog" doesn't monitor hardware heartbeats but the iteration hash of `loop-state.json`. But the pattern is the same:

- **Dispatch → Monitor → Detect failure → Escalate → Recover or fail gracefully**

This documentation walks through this pipeline, from input to final merge.

KEY

This is not a theoretical document. Every feature described here has been tested and developed on real production projects — wt-orchestrate runs daily on production codebases, autonomously, for hours, without human intervention.

Chapter 2

Why Is This Needed?

2.1 The Problem

AI-powered code development tools — like Claude Code, Cursor, or GitHub Copilot — have revolutionized individual developer productivity. A developer in a single session can implement complex features within hours.

But what if the task is **not one feature**, but *twenty*?

A mid-size specification (e.g., the next version of a SaaS application) typically contains 10-30 independent development tasks. There are dependencies between them: authentication is needed for the profile page, the API is needed for the dashboard, migrations are needed for every database operation. Managing this manually:

- A feature is done → manually merge, manually test
- Merge conflicts → manually resolve, re-test
- An agent gets stuck → manually debug, context-switch, restart
- Specification changes → manually re-plan, re-prioritize
- Token budget runs out → manually monitor, restart session

All of this is **management overhead**: a significant portion of the developer's time goes not to coding but to pipeline coordination.

2.2 The Solution: Autonomous Orchestration

`wt-orchestrate` automates this overhead. Starting from a single specification:

1. **Automatically plans** the tasks (decomposition, DAG)
2. **Executes them in parallel** in isolated worktrees
3. **Continuously monitors** the agents (15s poll, watchdog)
4. **Automatically tests and reviews** completed work (quality gates)
5. **Merges** the results (merge, conflict resolution)
6. **Continues** to the next phase (auto-replan)

This means that after handing off a specification, the system can work for *hours* without supervision, while the developer focuses on other tasks — or sleeps.

2.3 Who Is This For?

This system is built for **individual developers and small teams** working with AI agents. It doesn't replace CI/CD (Jenkins, GitHub Actions) — it complements it, in the *development phase*, before code even reaches the CI pipeline.

Typical usage pattern:

```
Morning:   Hand off spec → wt-orchestrate plan → start
Daytime:   Other tasks, meetings, planning
Evening:   wt-orchestrate status → 12/15 changes merged, 2 running, 1 failed
           Review → approve → the rest continues
Next day:  Everything done, open PR
```

KEY

The goal is not that humans "aren't needed." The goal is that humans are needed where humans are irreplaceable: writing the specification, making design decisions, and final approvals — not babysitting the pipeline.

2.4 How Is This Different from CI/CD?

Aspect	CI/CD (e.g., GitHub Actions)	wt-orchestrate
When it runs	After commit/PR	*Before* commit --- during development
What it does	Build, test, deploy	Planning, implementation, test, merge
Who works	Deterministic scripts	AI agents (creative, adaptive)
Error handling	Fail → red pipeline	Fail → retry, redispatch, escalate
Parallelism	Job matrix (fixed)	DAG-based, dynamic dispatch
Feedback loop	Human fixes, re-push	Agent fixes, re-tests

The two complement each other: `wt-orchestrate` produces the code, the CI/CD pipeline validates and deploys it.

Chapter 3

Development History

The first commit of `wt-tools` dates to January 9, 2026 — at that point it lived in a private, internal repository as a simple worktree manager. Public development restarted in February (earlier history isn't visible in the git log due to a force push). In January-February the worktree lifecycle took shape (`wt-new`, `wt-merge`, `wt-close`) along with the Ralph iterative loop. In February it became clear that manual coordination doesn't scale, and the first version of `wt-orchestrate` was born: plan generation, DAG, parallel dispatch. Live runs (the sales-raketa project) brought the third chapter: the watchdog system, the verify pipeline, token budget control, and `wt-sentinel` crash recovery. By March the spec digest pipeline, requirement coverage tracking, cascade failure handling, and phase-end E2E testing had matured. In the first week the system ran unsupervised for 5 minutes. Two months later it handles 5 hours — overnight, while sleeping, on production codebases.

KEY

The most important lesson: an orchestration system's value is not in handling the "happy path" — anyone can do that. The value is in error handling, recovery, and escalation. 80% of the system deals with what can go wrong.

Chapter 4

The wt-tools Ecosystem

4.1 Overview

`wt-orchestrate` doesn't stand alone — it's part of a broader toolkit covering the entire AI-assisted development workflow. The `wt-tools` package includes CLI tools, hooks, a memory system, and project templates.

4.2 Core Tools

4.2.1 Worktree Lifecycle

These tools manage the full lifecycle of git worktrees — from creation to merge and cleanup. Each agent works in an isolated worktree on its own branch, without disturbing the main branch.

Tool	Function
wt-new	Create new worktree and branch, .claude/ initialization
wt-list	List active worktrees, status summary
wt-status	Detailed agent state: PID, iteration, tokens, model
wt-merge	Merge worktree to main, LLM conflict resolution
wt-close	Worktree cleanup (branch delete, directory removal)
wt-focus	Switch focus: terminal focus to selected worktree
wt-add	Add existing branch as worktree

4.2.2 Agent Execution

The Ralph loop is the iterative development cycle controlled by `wt-loop`. The agent writes code, runs tests, and decides on next steps in each iteration. The loop automatically handles context window limits, token budgets, and API rate limits.

Tool	Function
wt-loop	Ralph iterative loop: max-turns, token budget, model routing
wt-work	Interactive work session in a worktree

Tool	Function
wt-skill-start	Skill-based session start in worktree

4.2.3 Orchestration and Supervision

The orchestration layer coordinates the entire pipeline — from specification processing to final merge. The sentinel is the supervisor that restarts the orchestrator on crash.

Tool	Function
wt-orchestrate	Full orchestration pipeline (the subject of this document)
wt-sentinel	Orchestrator supervisor: crash recovery, checkpoint handling
wt-manual	Manual orchestration intervention (debug, state editing)
wt-e2e-report	E2E test result aggregation and reporting

4.2.4 Memory and Context

The memory system ensures agents learn from previous sessions. Each project has its own memory database storing important decisions, lessons, and context. The hook system automatically injects relevant memories into the agent's context.

Tool	Function
wt-memory	Memory CLI: remember, recall, forget, search, sync
wt-hook-memory	Automatic memory injection (SessionStart, PostTool)
wt-hook-memory-warmstart	Session start: load relevant memories
wt-hook-memory-recall	Per-prompt topic-based memory recall
wt-hook-memory-posttool	Post-tool-use context augmentation
wt-hook-memory-save	Session-end memory save and extraction

4.2.5 Project Management

The `wt-project` system ensures new and existing projects can be set up with a single command — hooks, commands, skills installation, project registration. The `init` command is idempotent: re-running updates installed files without overwriting configuration.

Tool	Function
wt-project	Project initialization, hook deployment, registration
wt-config	Global and project-level configuration management
wt-deploy-hooks	Deploy Claude Code hooks to project
wt-version	Version display
wt-usage	Token usage statistics

4.2.6 Team Synchronization

Team sync enables communication between multiple developers (or agents) — messaging, activity sharing, memory synchronization. `wt-control` uses a git orphan branch as the communication channel.

Tool	Function
wt-control	Team sync: messaging, activity, coordination
wt-control-init	Team sync initialization (orphan branch)
wt-control-sync	Synchronize: push/pull messages and activity
wt-control-chat	Chat with other agents / developers
wt-control-gui	Graphical team dashboard
wt-hook-activity	Automatic activity sharing

4.2.7 OpenSpec and Quality

OpenSpec is the structured development workflow: from proposal through design and tasks to implementation. The audit and review tools ensure quality gates work not just in orchestration but in manual development too.

Tool	Function
wt-openspec	OpenSpec artifact management (proposal, design, tasks)
wt-audit	Code audit: security, quality, best practices

4.3 Project Templates

The `wt-project` system is template-based. Currently two templates are available, but the system is open to any technology stack:

4.3.1 wt-project-base

The base template usable for any project. Includes:

- Claude Code hooks (memory, activity, skill dispatch)
- `/wt:*` commands (orchestrate, decompose, help)
- OpenSpec skills (fast-forward, apply, verify)
- Sentinel rules
- Default `.claude/` configuration

4.3.2 wt-project-web (Next.js)

The web application template, extending `wt-project-base`. Includes:

- Next.js specific configurations
- Testing strategy (Jest + Playwright)
- Database handling (Prisma/Drizzle support)
- Pre-configured `smoke_command` and `e2e_command`
- Dev server management

4.3.3 Other Directions

The template system is intentionally open. `wt-project-web` shows the Next.js direction, but any technology stack can be supported with a custom template:

- **wt-project-api** — REST/GraphQL API backends
- **wt-project-mobile** — React Native, Flutter applications
- **wt-project-python** — Python/FastAPI/Django projects
- **wt-project-scraper** — Data collection and processing projects

These templates would build on `wt-project-base` and add project-specific configuration, testing strategy, and build pipelines.

4.4 Ecosystem Map

```
┌──────────────────────────────────────────────────────────┐
│                    wt-tools ecosystem                    │
├──────────────┬──────────────┬──────────────────────────┤
│  Worktree    │ Orchestration│  Memory & Context        │
│  lifecycle   │ & supervision│                          │
│              │              │                          │
│  wt-new      │ wt-orchestr. │  wt-memory               │
│  wt-list     │ wt-sentinel  │  wt-hook-memory-*        │
│  wt-status   │ wt-manual    │                          │
│  wt-merge    │ wt-e2e-rep.  │                          │
│  wt-close    │              │                          │
│  wt-loop     │              │                          │
├──────────────┼──────────────┼──────────────────────────┤
│  Project     │  Team Sync   │  OpenSpec & Quality      │
│  management  │              │                          │
│              │  wt-control  │  wt-openspec             │
│  wt-project  │  wt-ctrl-*   │  wt-audit                │
│  wt-config   │  wt-hook-act.│  /opsx:* skills          │
│  wt-deploy   │              │                          │
├──────────────┴──────────────┴──────────────────────────┤
│  Project templates                                      │
│  wt-project-base │ wt-project-web │ wt-project-...     │
└──────────────────────────────────────────────────────────┘
```

KEY

The ecosystem is modular: you don't have to use everything. The simplest entry point is wt-new + wt-loop (single agent, single worktree). Orchestration and the memory system can be enabled gradually, as needed.

Chapter 5

Overview

5.1 What Is Orchestration?

`wt-orchestrate` is an autonomous orchestration system that independently plans, executes, and verifies development tasks starting from a software specification. The entire process begins with a single command:

```
wt-orchestrate --spec docs/v3.md plan
wt-orchestrate start
```

The system then:

1. **Understands** the specification (digest, requirement identifiers)
2. **Plans** the execution (changes, dependency graph)
3. **Executes tasks in parallel** (worktrees, AI agents)
4. **Verifies** quality (test, review, verify gates)
5. **Merges** the results (merge, post-merge pipeline)
6. **Continues** to the next phase (auto-replan)

KEY

The orchestrator is not a single large LLM call. It is a state machine-based system that checks agent status every 15 seconds, handles errors, and makes autonomous decisions about how to proceed.

5.2 The 5-Layer Model

The orchestration consists of five layers that build upon each other:

Layer	Name	Responsibility	Key Modules
L1	Input	Specification reading, configuration resolution	config.sh, utils.sh
L2	Planning	Digest generation, decomposition, DAG	digest.sh, planner.sh
L3	Execution	Worktree dispatch, Ralph loop, monitoring	dispatcher.sh, monitor.sh, watchdog.sh

Layer	Name	Responsibility	Key Modules
L4	Quality	Test, review, verify, smoke, E2E gates	verifier.sh
L5	Delivery	Merge, post-merge, auto-replan	merger.sh, monitor.sh

L1 and L2 are the "thinking" phase: the system understands the task and plans the approach. L3 is the "work" layer where AI agents actually write code — this is where most time and tokens are spent. L4 is "quality assurance," ensuring the completed work is actually good. L5 is "delivery": the result enters the main branch and the system decides whether to continue.

5.3 Modular Architecture

`wt-orchestrate` has a single entry point (`bin/wt-orchestrate`), but the implementation is split across 14 source modules under `lib/orchestration/`:

```
bin/wt-orchestrate          ← entry point, CLI parsing
lib/orchestration/
├── events.sh               ← JSONL event log
├── config.sh               ← configuration resolution
├── utils.sh                ← helper functions
├── state.sh                ← state management (JSON)
├── orch-memory.sh          ← memory integration
├── watchdog.sh             ← stall detection, escalation
├── planner.sh              ← decomposition, validation
├── builder.sh              ← base build health
├── dispatcher.sh           ← change lifecycle
├── verifier.sh             ← quality gates
├── merger.sh               ← merge, cleanup, archive
├── digest.sh               ← spec digest, coverage
├── reporter.sh             ← HTML report generation
└── monitor.sh              ← main monitor loop
```

Module loading order matters: `events.sh` is loaded first (every other module emits events), then `state.sh`, then the rest.

5.4 State Files

The orchestrator stores its state in three main files:

File	Contents
orchestration-plan.json	The decomposition plan (changes, DAG, requirements)
orchestration-state.json	Runtime state (change statuses, token counts, merge queue)
orchestration-summary.md	Human-readable summary

These files are created in the project root and are not version-controlled (they are in `.gitignore`).

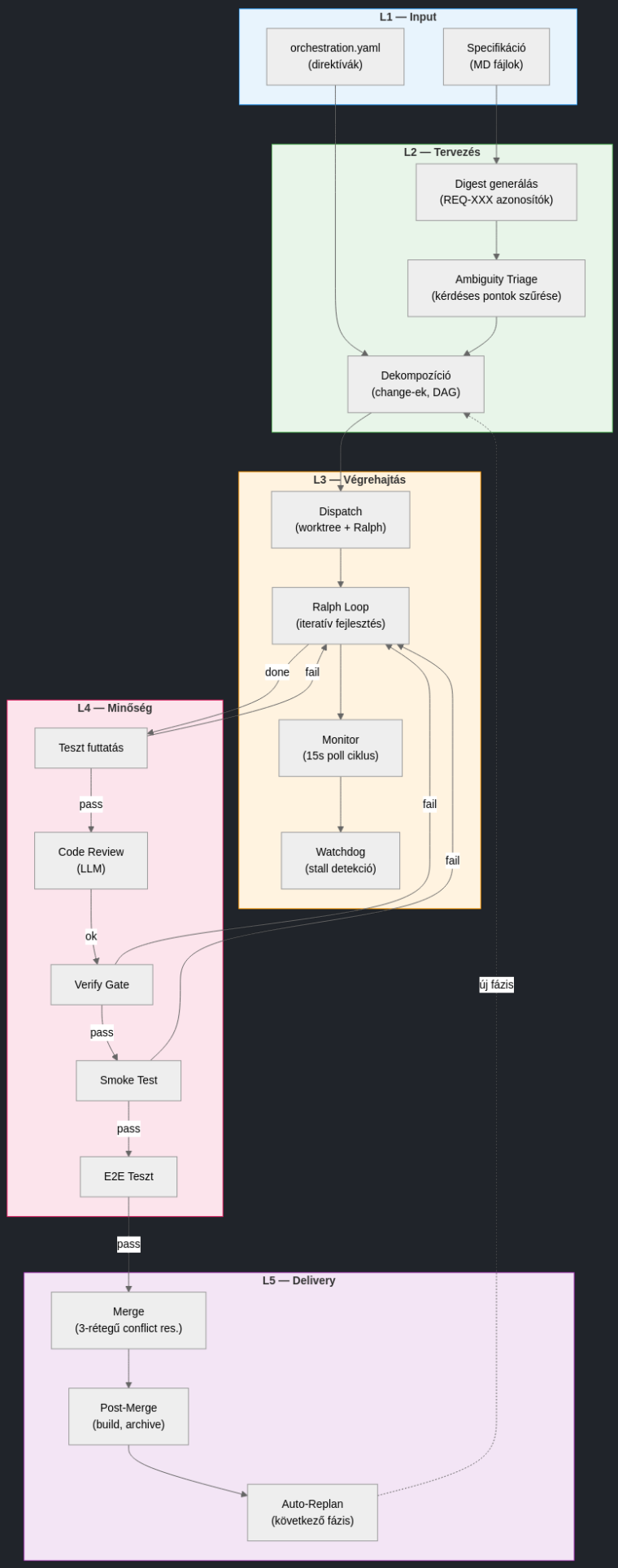

Figure 5.1: The complete orchestration pipeline overview

5.5 A Typical Run

A typical run of a medium project (10-15 changes, 3 parallel agents):

1. **Plan** (1-2 minutes): Spec processing, digest, decomposition
2. **Dispatch + Execution** (30-120 minutes): Parallel development in worktrees
3. **Verification** (continuous): Test + review for every completed change
4. **Merge** (continuous): Merge to main branch according to checkpoint policy
5. **Replan** (optional): If there is a next phase, replan and continue

The full pipeline can run for hours without supervision, while the watchdog and monitor loop handle error recovery.

Chapter 6

Input and Configuration

6.1 Input Modes

The orchestrator accepts three types of input. These are mutually exclusive.

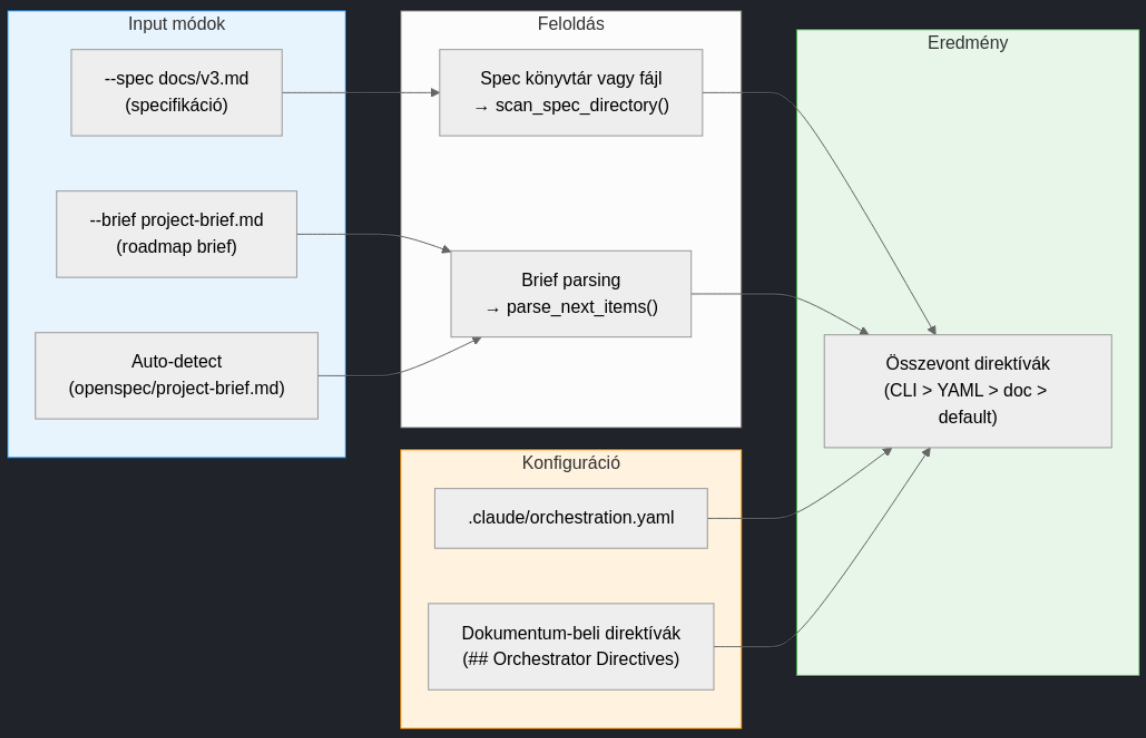

Figure 6.1: Input modes and configuration resolution

6.1.1 1. Specification Mode (--spec)

Most projects work with specification documents:

```
wt-orchestrate --spec docs/v3.md plan
wt-orchestrate --spec v12 plan          # shortname: wt/orchestration/specs/v12.md
wt-orchestrate --spec docs/ plan         # entire directory
```

`--spec` can point to a file or directory. For directories, the system processes all `.md` files, and the `scan_spec_directory()` function builds the complete picture.

Phase filtering: The `--phase` option narrows to a specific phase:

```
wt-orchestrate --spec docs/v3.md --phase 2 plan    # phase 2 only
wt-orchestrate --spec docs/v3.md --phase "Security" plan  # text filter
```

6.1.2 2. Brief Mode (--brief)

The earlier, simpler format uses a `### Next` section to list roadmap items:

```
## Feature Roadmap
### Next
- Auth system: JWT-based authentication
- User profile: Profile editing and avatar
### Later
- Admin panel: User management
```

6.1.3 3. Auto-detect

If neither `--spec` nor `--brief` is specified, the system searches automatically:

1. `openspec/project-brief.md` — if it contains `### Next` items
2. `openspec/project.md` — fallback
3. Error if neither is found

6.2 Configuration

Behavior can be configured at three levels. Higher levels override lower ones:

```
CLI flag  >  orchestration.yaml  >  document directives  >  defaults
```

6.2.1 orchestration.yaml

The main configuration file is `.claude/orchestration.yaml` (or `wt/orchestration.yaml`):

```yaml
# Execution
max_parallel: 3               # max parallel changes
merge_policy: checkpoint       # eager | checkpoint | manual
checkpoint_every: 3            # checkpoint after N changes

# Testing
test_command: "pnpm test"      # test command
test_timeout: 300              # test timeout (seconds)
smoke_command: "pnpm build"    # smoke test command
smoke_blocking: true           # does smoke fail block merge

# Models
default_model: opus            # implementation model
review_model: sonnet           # review model
summarize_model: haiku         # summarization model
```

```yaml
# Review
review_before_merge: true      # LLM review before merge
max_verify_retries: 2          # verify retry limit

# Automation
auto_replan: true              # auto-plan for next phase
context_pruning: true          # context window optimization
model_routing: "off"           # model routing strategy

# Safety
token_budget: 5000000          # soft limit (warning)
token_hard_limit: 20000000     # hard limit (human approval)
time_limit: "5h"               # runtime limit

# Hooks
post_merge_command: "pnpm db:generate"
hook_pre_dispatch: ""
hook_post_verify: ""
hook_pre_merge: ""
hook_post_merge: ""
hook_on_fail: ""
```

6.2.2 Document Directives

Directives can also be specified within the specification itself:

```
## Orchestrator Directives
- max_parallel: 4
- merge_policy: eager
- token_budget: 100000
```

The system reads these values using the `parse_directives()` function.

> **KEY**
> CLI flags always win. If `--max-parallel 5` is on the command line, it overrides both YAML and document directives.

6.3 Directive Reference

Directive	Type	Default	Description
max_parallel	number	3	Max parallel worktrees
merge_policy	enum	checkpoint	eager/checkpoint/manual
checkpoint_every	number	3	Checkpoint frequency
test_command	string	""	Test command
test_timeout	number	300	Test timeout (seconds)
smoke_command	string	""	Smoke test command

Directive	Type	Default	Description
smoke_blocking	bool	true	Does smoke fail block merge
smoke_fix_max_retries	number	3	Smoke fix retry limit
e2e_command	string	""	E2E test command
e2e_mode	enum	per_change	per_change/phase_end
default_model	string	opus	Implementation model
review_model	string	sonnet	Review model
review_before_merge	bool	false	LLM review enabled
max_verify_retries	number	2	Verify retry limit
auto_replan	bool	false	Auto-replan at phase end
token_budget	number	0	Soft token limit
token_hard_limit	number	20M	Hard token limit
time_limit	string	5h	Runtime limit
watchdog_timeout	number	600	Watchdog timeout (seconds)
context_pruning	bool	true	Context optimization
model_routing	string	off	Model routing strategy
post_merge_command	string	""	Post-merge command

Chapter 7

Digest and Triage

7.1 What Is a Digest?

A digest is a structured extract of the specification. It breaks a multi-thousand-line spec document into machine-processable JSON format, where every requirement receives a unique identifier (REQ-001, REQ-002, etc.).

This enables the orchestrator to:

- **Track** which requirement is implemented by which change
- **Verify** that every requirement is covered
- **Identify** ambiguous points before development begins

7.2 Digest Generation Steps

7.2.1 1. Spec Scanning (scan_spec_directory)

The system maps the input file or directory:

```
wt-orchestrate --spec docs/specs/ digest
```

Scan results:

- **file_count**: number of files processed
- **source_hash**: hash computed from all file contents (freshness check)
- **master_file**: if there is an `index.md` or similar, it becomes the main file

7.2.2 2. Prompt Assembly (build_digest_prompt)

The system builds a structured prompt for the Claude API, containing all spec files and the expected output format.

7.2.3 3. API Call and Parsing

Claude processes the specification and returns structured JSON output:

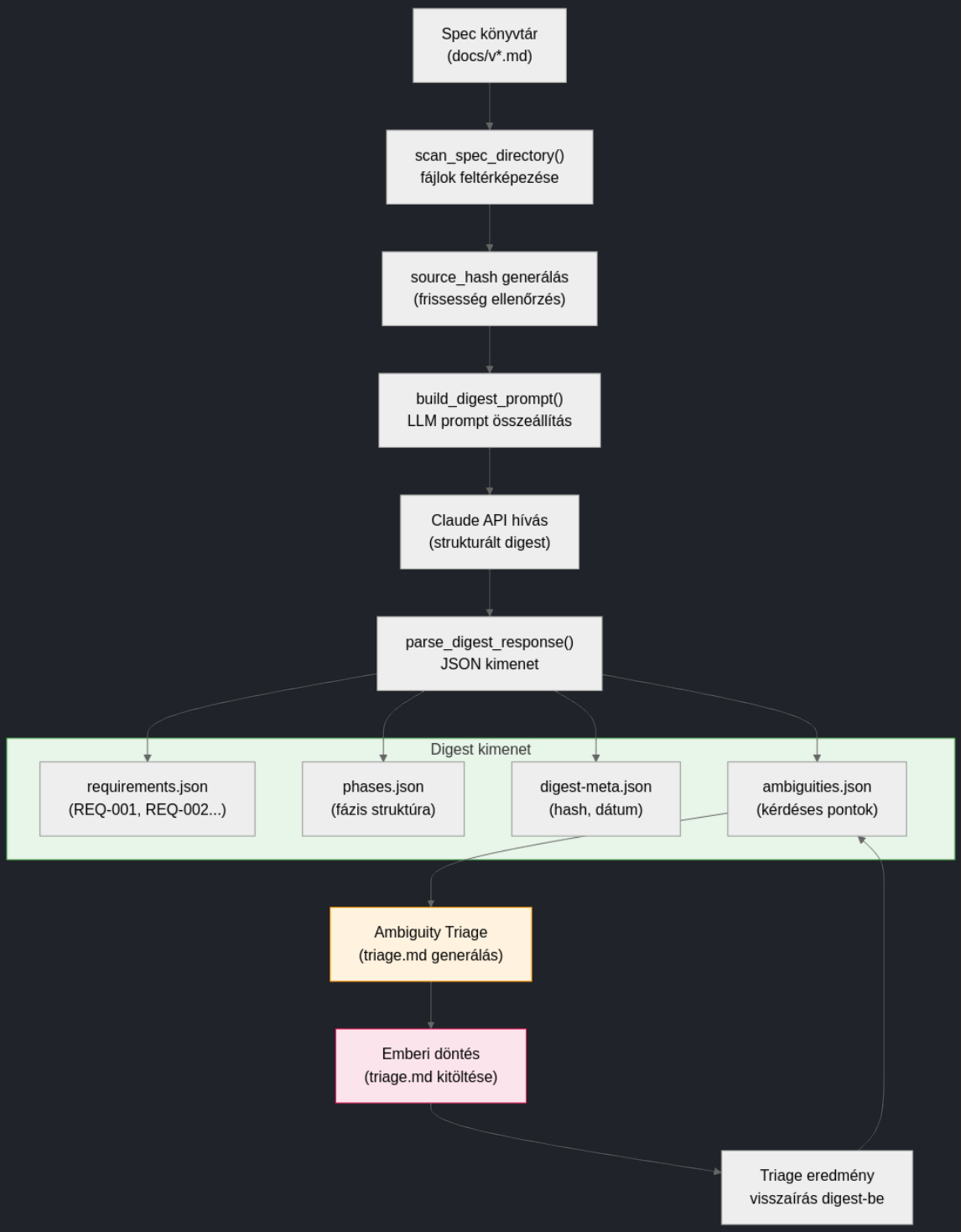

Figure 7.1: The complete digest generation flow

```
{
  "requirements": [
    {
      "id": "REQ-001",
      "title": "JWT authentication",
      "brief": "Token-based auth on /api/* endpoints",
      "priority": "high",
      "source_file": "docs/v3-security.md",
      "section": "2.1 Authentication"
    }
  ],
  "phases": [...],
  "ambiguities": [...]
}
```

7.2.4 4. ID Stabilization (stabilize_ids)

If a previous digest exists, the system preserves existing REQ-XXX identifiers so that requirement tracking doesn't break after an update.

7.3 Digest Output Files

The digest writes to the `wt/orchestration/digest/` directory:

File	Contents
requirements.json	All requirements with REQ-XXX identifiers
phases.json	Phase structure (what belongs to which phase)
digest-meta.json	Hash, date, file counts
ambiguities.json	Ambiguous or incomplete points

7.4 Freshness Check

The `source_hash` field enables quick freshness verification:

```
# If the hash hasn't changed, the digest is up-to-date
if [[ "$current_hash" == "$stored_hash" ]]; then
    echo "Digest up-to-date, skipping"
fi
```

This prevents unnecessary regeneration when the spec hasn't changed.

7.5 Ambiguity Triage

Digest generation automatically identifies ambiguous points in the specification. These are handled through the **triage** process.

7.5.1 Triage Generation

The system generates a `triage.md` file that lists ambiguous points and offers decision options:

```
## AMB-001: Unclear session handling

**Context**: The spec mentions "session timeout" but doesn't define the value.

**Options**:
- [ ] A: 30 minutes (web standard)
- [ ] B: 60 minutes (longer session)
- [ ] C: Configurable (runtime setting)

**Decision**: ___
```

7.5.2 Human Decision

A human (or the sentinel) fills in the triage file. The decision is written back to `ambiguities.json`, and the planner takes it into account during decomposition.

7.5.3 Automatic Triage Merging

The `merge_triage_to_ambiguities()` and `merge_planner_resolutions()` functions ensure that:

- Human decisions are incorporated into the digest
- Planner-made decisions are also preserved
- In case of conflict, the human decision wins

KEY

The triage gate is the only point where the orchestrator stops and asks for human intervention. Every other decision is made autonomously. If there are no ambiguous points, the triage step is automatically skipped.

7.6 Requirement Coverage

The digest enables requirement coverage tracking throughout the entire pipeline:

```
wt-orchestrate coverage
```

Output:

```
Requirement Coverage: 12/15 (80%)
  ✓ REQ-001: JWT authentication      → change/auth-system (merged)
  ✓ REQ-002: User profile            → change/user-profile (running)
  ✗ REQ-003: Admin panel             → (not assigned)
  ...
```

The `update_coverage_status()` function automatically updates coverage when a change's status changes (merged, failed, etc.).

Chapter 8

OpenSpec --- The Spec-Driven Workflow

8.1 What Is OpenSpec?

OpenSpec is a structured development workflow that guides every change through phases before implementation begins. The goal: the AI agent should not "just code" but *understand* the task and make traceable decisions.

The phases:

```
Proposal (WHY) → Design (HOW) → Specs (WHAT) → Tasks (DO) → Implementation
```

8.2 The Artifacts

8.2.1 Proposal

The *why* document: motivation, scope, success criteria. The orchestrator **pre-generates** the proposal in the worktree from the plan's roadmap item and scope — the agent receives this as a starting point.

8.2.2 Design

The *how* document: technical decisions in ADR (Architecture Decision Record) format, risks, alternatives. Generated by the agent from the proposal.

8.2.3 Specs

The *what* document: detailed requirements in BDD (Given/When/Then) format, delta operations (ADDED, MODIFIED, REMOVED). Every requirement has testable scenarios.

8.2.4 Tasks

The *do* list: a checkbox task list that the agent checks off one by one during implementation. The `[?]` marker indicates a manual (human) task — API key procurement, external service setup.

```
- [x] Add JWT middleware to /api/* routes
- [x] Create login endpoint with bcrypt
```

```
- [ ] Add refresh token rotation
- [?] Configure OAuth provider credentials
```

8.3 Integration with Orchestration

8.3.1 Dispatch Phase

`dispatcher.sh` during worktree creation:

1. Creates the change directory (`openspec new change`)
2. **Pre-generates the proposal** from the plan scope
3. Copies spec context (in digest mode)
4. Appends cross-cutting file list, sibling change statuses, and memory context

8.3.2 Ralph Loop Integration

The Ralph loop decides based on OpenSpec artifact state:

```
If no tasks.md → action: ff (fast-forward, create all artifacts)
If tasks.md exists → action: apply (implementation)
If everything done → action: done
```

In the `ff` phase, the agent runs the `/opsx:ff` skill: creates design, specs, and tasks from the proposal in a single session. In the `apply` phase, it runs `/opsx:apply`: implements tasks from tasks.md.

8.3.3 FF→Apply Chaining

Previously, artifact generation (ff) and implementation (apply) ran in separate iterations, and the iteration boundary caused context loss. The current version chains them within a single iteration: after ff completes, apply starts immediately with fresh context.

8.4 Interactive vs. Orchestrated Usage

Mode	Commands	When
Interactive	/opsx:new → /opsx:continue → /opsx:apply → /opsx:verify → /opsx:archive	Developer working alone, step by step
Fast-forward	/opsx:ff → /opsx:apply → /opsx:archive	Developer, but no time for step-by-step
Orchestrated	Automatic --- dispatcher and Ralph loop handle it	wt-orchestrate start

8.5 Traceability

OpenSpec provides a complete traceability chain:

```
Spec section (v3.md)
  → Requirement ID (REQ-042)
```

```
→ Change assignment (platform-dashboard)
  → Proposal: "This change covers REQ-042"
    → Tasks: "Add KPI cards section"
      → Git commit: code implementing REQ-042
```

The verify gate checks not just code quality but **requirement compliance**: the review prompt includes the assigned REQ-XXX identifiers.

KEY

OpenSpec is the contract between orchestration and execution. The orchestrator creates the proposal; the agent creates the design, specs, and tasks; the verify gate checks the implementation. If any phase is missing, the system knows where it stands and what the next step is.

8.6 Limitations

- **FF failures**: `/opsx:ff` sometimes fails to create tasks.md (large spec, truncated LLM response). In such cases, the loop generates fallback tasks from the proposal
- **Manual tasks**: Tasks marked with `[?]` (API keys, external setup) require human intervention — orchestration cannot automate them
- **Large changes**: 14+ requirements in a single change → unreliable. The planner applies a max 6 REQ/change rule

Chapter 9

Planning

9.1 The Purpose of Decomposition

The planning phase breaks the specification (or digest) into **executable changes**. Each change is a self-contained, well-defined development task that an AI agent executes in a worktree.

9.2 Planning Methods

Two planning methods exist, controlled by the `plan_method` directive:

9.2.1 API Mode (default)

A single Claude API call performs the decomposition:

```
wt-orchestrate --spec docs/v3.md plan   # plan_method: api (default)
```

Advantages:

- Fast (1-2 minutes)
- Low token usage
- Deterministic output

9.2.2 Agent Mode

A dedicated worktree runs a Ralph loop for planning, across multiple iterations:

```yaml
# orchestration.yaml
plan_method: agent
plan_token_budget: 500000
```

This method is useful when:

- The specification is very complex
- The planner needs to read code for good decomposition
- Test infrastructure discovery is needed

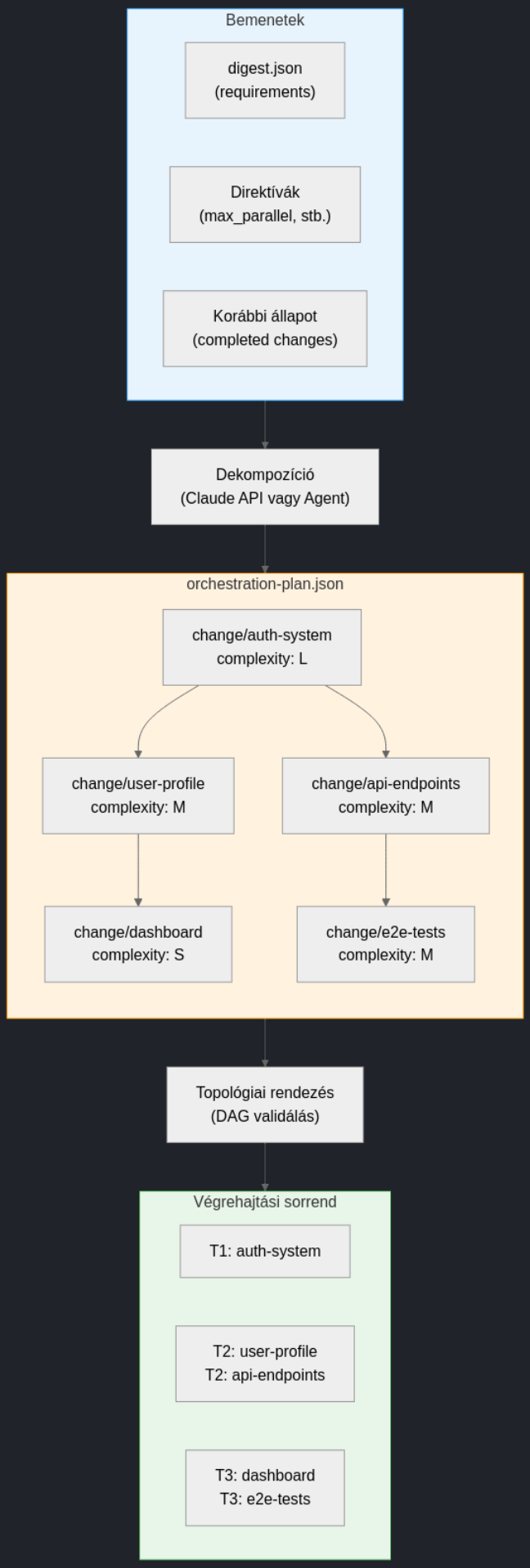

Figure 9.1: Decomposition and DAG generation

9.3 The Plan Structure

The result of `cmd_plan()` is `orchestration-plan.json`:

```
{
  "plan_version": 1,
  "brief_hash": "a1b2c3d4",
  "plan_phase": "phase-2",
  "plan_method": "api",
  "changes": [
    {
      "name": "auth-system",
      "scope": "Implement JWT authentication on /api/* endpoints",
      "complexity": "L",
      "change_type": "feature",
      "depends_on": [],
      "roadmap_item": "Auth system",
      "requirements": ["REQ-001", "REQ-004"],
      "also_affects_reqs": ["REQ-012"],
      "model": null,
      "skip_review": false,
      "skip_test": false
    },
    {
      "name": "user-profile",
      "scope": "Profile editing and avatar upload",
      "complexity": "M",
      "depends_on": ["auth-system"],
      "requirements": ["REQ-002", "REQ-003"]
    }
  ]
}
```

9.3.1 Field Descriptions

Field	Description
name	Unique identifier (becomes the branch name: change/auth-system)
scope	Detailed description for the agent
complexity	S (small), M (medium), L (large) --- determines token limit
change_type	feature, fix, refactor, test, docs, config
depends_on	Dependencies (other change names)
requirements	Assigned REQ-XXX identifiers
also_affects_reqs	Cross-cutting requirements (awareness)
model	Change-specific model override (null = default)
skip_review	Skip review gate (e.g., for config changes)
skip_test	Skip test gate (e.g., for docs changes)

9.4 Topological Sort (DAG)

The `topological_sort()` function builds a directed acyclic graph (DAG) from the `depends_on` fields and returns changes in topological order.

Rules:

1. Changes without dependencies can start in parallel
2. A change can only start when all its dependencies are in `merged` status
3. Circular dependencies result in an error during plan generation

Example: If A ← B and A ← C, but D ← B,C:

```
T1: A (no dependencies)
T2: B, C (in parallel, after A merged)
T3: D (after B and C merged)
```

9.4.1 Circular Dependency Detection

```
# The self-test includes a test:
wt-orchestrate self-test
# → PASS: circular dependency detected
```

If the planner generates circular dependencies, the system throws an error and the plan must be rejected.

9.5 Test Infrastructure Discovery

The `detect_test_infra()` function assesses the project's testing capabilities during the plan phase:

- **Framework**: vitest, jest, pytest, mocha
- **Config files**: vitest.config.ts, jest.config.js, etc.
- **Test files**: `*.test.*`, `*.spec.*`, `test_*.py`
- **Helper directories**: `src/test/`, `__tests__/`

This information is included in the planner prompt so that change scopes include testing.

9.6 Spec Summarization for Large Documents

If the spec is too large (many thousand tokens), `summarize_spec()` uses a haiku model to summarize the material before the planner receives it:

1. The summarizer extracts the phase structure
2. Identifies completed and missing parts
3. The planner receives only the relevant phase

KEY

The plan is always viewable after generation: `wt-orchestrate plan --show`. Before issuing the `start` command, it's worth checking that the decomposition is sensible.

9.7 Plan Validation

The generated plan is automatically validated:

- Every change name must be unique
- `depends_on` references must point to existing changes
- Circular dependencies are not allowed
- `complexity` values must be S, M, or L

If validation fails, the plan is not saved and the user receives an error message.

Chapter 10

Execution

10.1 Dispatch: The Change Begins

When all of a change's dependencies are satisfied (merged status), `dispatch_ready_changes()` starts execution. The process:

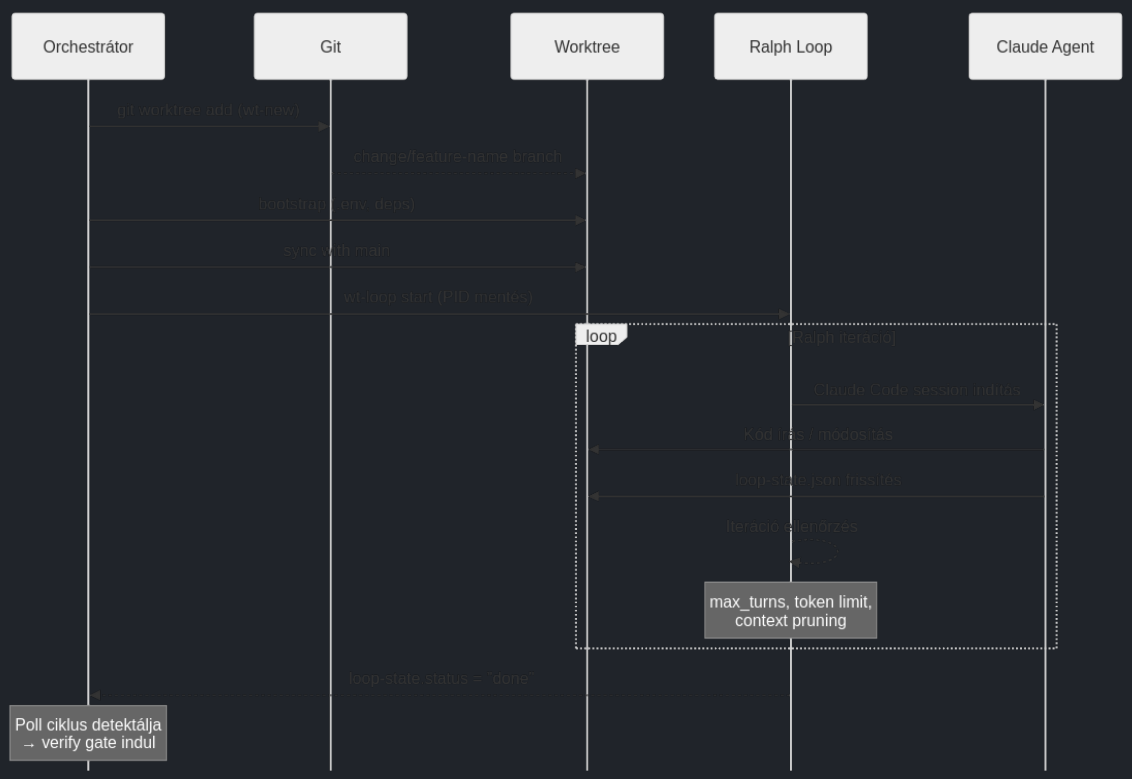

Figure 10.1: The complete dispatch and Ralph loop flow

10.1.1 1. Worktree Creation

```
wt-new <change-name>
# → .claude/worktrees/<change-name>/
# → branch: change/<change-name>
```

`wt-new` creates an isolated git worktree where the agent can work freely without affecting the main

branch.

10.1.2 2. Bootstrap

`bootstrap_worktree()` ensures the worktree is operational:

- Copying `.env` files from the project root
- Installing dependencies (if `package-lock.json` is newer)
- `.env.local`, `.env.development` files are also copied

10.1.3 3. Synchronization with Main Branch

`sync_worktree_with_main()` ensures the worktree is built on the latest main:

- If the worktree is up-to-date → skip
- If there's a difference → `git merge main` into the worktree branch
- Generated file conflicts (lock file, `.tsbuildinfo`) → automatic `--ours` resolution
- Real code conflicts → merge abort, warning

10.1.4 4. Ralph Loop Start

```
wt-loop start --change <name> --model <model_id> --max-turns <N>
```

The Ralph PID is saved to the state file.

10.2 The Ralph Loop

The Ralph loop starts with the `wt-loop` command and executes an iterative development cycle:

10.2.1 Iteration Cycle

In each iteration:

1. A **Claude Code session** starts in the worktree
2. The agent receives the **scope** (task description) and optional **retry context**
3. The agent **writes code**, runs tests, modifies files
4. `loop-state.json` is updated at the end of the iteration

10.2.2 loop-state.json

The iteration state is stored in a JSON file:

```
{
  "status": "running",           // running | done | error
  "iteration": 5,
  "max_turns": 20,
  "model": "claude-opus-4-6",
  "started_at": "2026-03-10T14:00:00Z",
  "tokens_used": 450000,
  "input_tokens": 300000,
```

```
  "output_tokens": 150000,
  "cache_read_tokens": 120000,
  "last_activity": "2026-03-10T14:15:30Z"
}
```

10.2.3 Completion Conditions

The Ralph loop stops in the following cases:

Condition	Result
Agent signals: done	status: "done"
max_turns reached	status: "done" (forced)
Token limit reached	status: "done"
Fatal error	status: "error"

10.2.4 Context Pruning

The `context_pruning: true` directive (default) enables context window optimization:

- Output from the agent's previous iterations is summarized
- Only the essential parts remain instead of the full conversation
- This reduces token usage and improves quality

10.3 Model Routing

The `model_routing` directive allows different changes to use different models:

- **`off`** (default): every change uses the `default_model`
- Change-specific override in the plan: `"model": "sonnet"`

Typical configuration:

```
default_model: opus        # complex features
review_model: sonnet       # code review (faster, cheaper)
summarize_model: haiku     # summarization (fastest)
```

10.3.1 Complexity-Based Routing

The planner indicates change size with the `complexity` field:

Complexity	Characteristics	Typical Model
S (Small)	Config, docs, simple fix	sonnet
M (Medium)	Feature, refactor	opus
L (Large)	Architecture, cross-cutting	opus

10.4 Parallel Execution

The `max_parallel` directive controls how many worktrees can have active agents simultaneously:

```
max_parallel: 3

T=0    [auth-system] ──────────────────────────── done → verify
T=0    [api-endpoints] ──────────────────── done → verify
T=0    [config-update] ─── done → verify → merge
T=15   [user-profile] ─────────────────── (dispatch after auth merged)
T=30   [dashboard] ──────────── (dispatch after slot free)
```

The dispatch logic:

1. Count the `running` + `dispatched` + `verifying` changes
2. If fewer than `max_parallel` → find the next ready change
3. A change is "ready" if: `pending` status AND every `depends_on` entry is `merged`

KEY

Parallel execution doesn't mean all changes run simultaneously. The DAG and max_parallel together determine the execution schedule. A 15-change plan with parallelism of 3 typically runs in 5-6 "waves."

10.5 Pause and Resume

Execution can be suspended and resumed at any time:

```
wt-orchestrate pause auth-system    # one change
wt-orchestrate pause --all          # all changes
wt-orchestrate resume auth-system   # resume
wt-orchestrate resume --all         # resume all
```

`pause` stops the Ralph loop (PID kill), but the worktree remains. `resume` restarts the loop, optionally with retry context.

10.6 Token Tracking

Each change's token usage is tracked in real time:

- `tokens_used`: total tokens (input + output)
- `input_tokens`, `output_tokens`: detailed breakdown
- `cache_read_tokens`, `cache_create_tokens`: prompt cache statistics

The `_prev` fields store totals from previous iterations (counting remains accurate after restarts).

Chapter 11

The Sentinel

11.1 Why Does the Supervisor Need a Supervisor?

The orchestrator itself is a process — and processes sometimes die. OOM kill, API timeout, unexpected exception, broken pipe. If the orchestrator runs overnight without supervision and stops, the developer finds a half-finished state in the morning.

`wt-sentinel` answers this problem: a supervisor that watches the orchestrator, and if it stops, tries to restart it. It operates in two modes: **bash mode** (cost-free, deterministic) and **agent mode** (LLM-based, intelligent decision-making).

11.2 Bash Sentinel

The bash sentinel is a standalone script that wraps the `wt-orchestrate start` command:

```
wt-sentinel --spec docs/v3.md --time-limit 5h
# ↑ all options are passed through to the orchestrator
```

11.2.1 Crash Recovery with Exponential Backoff

When the orchestrator stops, the sentinel doesn't restart immediately — it waits with exponential backoff:

```
Crash 1: wait 30s  (+ 0-25% jitter)
Crash 2: wait 60s
Crash 3: wait 120s
Crash 4: wait 240s (maximum)
Crash 5: give up → SENTINEL_FAILED event
```

If the orchestrator ran for at least 5 minutes (sustained run), the counter and backoff reset. This distinguishes startup errors (configuration problem that always kills immediately) from runtime crashes (API timeout that happens once and then runs fine).

11.2.2 Liveness Detection

The sentinel doesn't parse logs — it watches the events file modification time (mtime):

```
Orchestrator → WATCHDOG_HEARTBEAT (every 15s) → events.jsonl mtime updates
Sentinel → polls mtime (every 10s)
  → If >180s since last change → orchestrator is stuck
  → SIGTERM → 30s wait → SIGKILL if needed → restart
```

The `WATCHDOG_HEARTBEAT` event is emitted by the monitor loop every cycle. If it's missing for more than 3 minutes, the sentinel intervenes.

11.2.3 State Recovery

Before every restart, the sentinel attempts to repair the state:

1. **Event-based reconstruction**: If `orchestration-state.json` is older than the events log, the sentinel reconstructs state from event replay — which change was where, how many tokens were used
2. **Dead PID detection**: Checks PIDs of `running` changes. If the process is no longer alive, the change moves to `stalled` status (the watchdog will handle it)
3. **State normalization**: If orchestrator status is `running` but no process exists, sets it to `stopped`

11.2.4 Exit Classification

The sentinel distinguishes between **terminal** and **transient** exits:

State	Category	Sentinel Action
done	Terminal	Stops, no restart
stopped	Terminal	Stops
time_limit	Terminal	Stops
plan_review	Terminal	Stops (human decision needed)
Any crash	Transient	Restart with backoff

11.3 Agent Sentinel

The `/wt:sentinel` skill runs as a Claude Code session and makes intelligent decisions:

11.3.1 Tiered Intervention

The agent sentinel operates on this principle: **don't intervene in orchestration-level problems** — the orchestrator has built-in recovery.

Tier	When	Examples
Tier 1 --- Don't intervene	Orchestration-level problem	merge-blocked, test fail, verify retry, replan
Tier 2 --- Intervene	Process-level problem	Crash, hang, terminal state, checkpoint

Merge conflict? The orchestrator handles it with 3-layer conflict resolution. Test failure? The orchestrator retries. Individual change failed? The orchestrator continues with the rest. The agent sentinel only steps in when the orchestrator process itself is stuck.

11.3.2 Checkpoint Handling

- **Periodic checkpoint** (after N merges): automatic approval
- **Budget checkpoint** (token hard limit): escalate to user
- **Other checkpoints**: user decision needed

11.3.3 Autonomy Rules

The sentinel operates with special rules:

- **Never ask before fixing** — if there's a bug, fix it, commit it, restart
- **Never ask before restarting** — after a crash, cleanup and restart without confirmation
- **Polling must never stop on its own** — after a fix, continue; after a restart, continue; after context compact, continue

KEY

The sentinel provides dual protection: the bash sentinel watches the process cost-free and restarts automatically, while the agent sentinel analyzes the situation with LLM and makes intelligent decisions. For production runs, the bash sentinel is the baseline; the agent sentinel is optional, useful for E2E testing and development.

11.4 Sentinel Events

The sentinel writes directly to the events JSONL (no dependency on the `events.sh` module):

Event	Meaning
SENTINEL_RESTART	Orchestrator stopped, restarting (exit code, backoff, crash count)
SENTINEL_FAILED	5 rapid crashes, sentinel gave up
STATE_RECONSTRUCTED	State reconstructed from events

Chapter 12

Monitor and Watchdog

12.1 The Monitor Loop

The heart of the orchestrator is the `monitor_loop()` function, which runs indefinitely after the `start` command is issued. Its job: check the status of all active changes every 15 seconds and take the appropriate action.

12.1.1 The Poll Cycle

In every 15-second cycle, the following occurs:

1. **Status check**: Inspect orchestrator status (running/paused/stopped)
2. **Active change polling**: Call `poll_change()` for every `running` and `verifying` change
3. **Watchdog check**: Call `watchdog_check()` for stall detection
4. **Suspended changes**: Check whether `paused` or `budget_exceeded` changes have finished
5. **Token budget**: Soft and hard limit checks
6. **Verify-failed recovery**: Retry if the retry limit allows
7. **Dispatch**: Start new changes if slots are available
8. **Merge queue**: Process the merge queue
9. **Stall recovery**: Restart stalled changes
10. **Cascade failure**: Propagate failed dependencies
11. **Replan**: If everything is done and `auto_replan` is active, next phase

12.1.2 poll_change()

`poll_change()` is the most important function. For a single change:

- Reads the `loop-state.json` file from the worktree
- Updates token counters in the state file
- If `status == "done"` → starts the verify pipeline (test → review → verify → smoke → E2E)
- If `status == "error"` → retry or fail

12.1.3 Active Time Tracking

The monitor doesn't measure wall clock time, but **active time**:

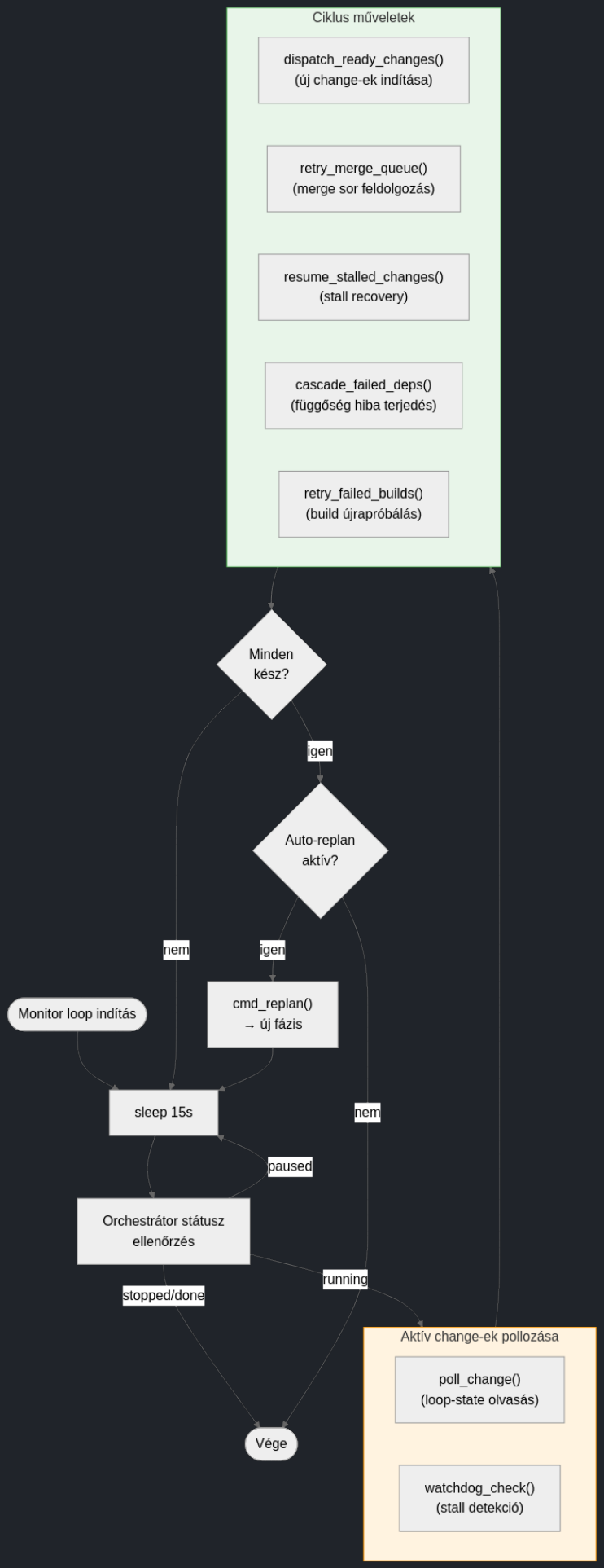

Figure 12.1: The monitor loop cycle

- Only counts when at least one Ralph loop is running (`any_loop_active()`)
- Doesn't count during token budget waits
- Doesn't count during pause
- Active time accumulates across restarts

```
Wall time:    |▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓|  3 hours
Active time:  |▓▓▓▓▓▓▓▓  ▓▓▓▓▓▓▓▓    ▓▓▓▓▓▓▓▓▓▓▓▓|  2 hours
               ^running  ^pause      ^running
```

12.2 The Watchdog System

The watchdog's job is detecting and handling "stuck" changes. Every active change has its own watchdog state.

12.2.1 Detection Mechanisms

12.2.1.1 1. Timeout Detection

Different timeouts per status:

Status	Default Timeout	Description
running	600s (10 min)	During implementation
verifying	300s (5 min)	During verify pipeline
dispatched	120s (2 min)	After dispatch, before Ralph starts

If more time has elapsed since the last activity than the timeout, escalation begins.

12.2.1.2 2. Action Hash Loop Detection

The watchdog maintains a "hash ring" for every change:

```
{
  "action_hash_ring": ["abc123", "abc123", "abc123", "abc123", "abc123"],
  "consecutive_same_hash": 5
}
```

The action hash is computed from key fields of `loop-state.json` (iteration number, token count, status). If the hash is identical N consecutive times (default: 5) → the agent is stuck.

12.2.1.3 3. Artifact Creation Grace Period

Between dispatch and Ralph loop start, there is a "grace period": the agent is creating OpenSpec artifacts (proposal, design, specs, tasks) and there is no `loop-state.json` yet. The watchdog recognizes this and does not escalate.

12.2.2 Escalation Levels

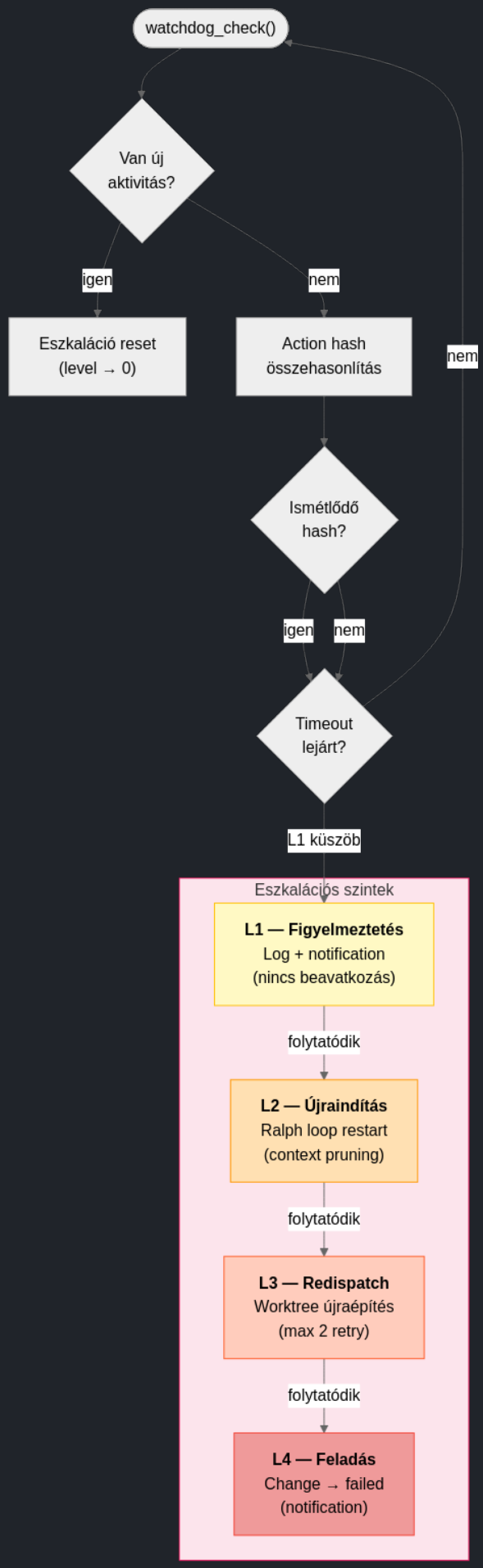

Figure 12.2: Watchdog escalation levels

Level	Action	Description
L1	Warning	Log entry + notification. No intervention.
L2	Restart	Stop and restart the Ralph loop. Context pruning activates.
L3	Redispatch	Complete worktree rebuild. Max max_redispatch (default: 2) times.
L4	Give up	Change moves to failed status. Notification sent.

The escalation levels represent gradual intervention. L1 only watches — perhaps the agent will solve the problem on its own (e.g., a long build running). L2 restarts the loop with fresh context — this is the most common recovery, successful about 70% of the time. L3 starts everything from scratch: fresh worktree, fresh branch, full redispatch — expensive, but if L2 doesn't help, this is the last chance. L4 gives up the fight: better to lose one change than burn infinite tokens.

12.2.3 Recovery Detection

If an escalated change shows activity again (e.g., the hash changes after an L2 restart), the escalation **automatically resets**:

```
L2 (restart) → activity detected → level reset → L0
```

KEY

The watchdog is the only safety net that prevents a stuck agent from consuming tokens indefinitely. In production runs, the watchdog_timeout and max_redispatch values should be tuned to the project's characteristics.

12.3 Token Safety Nets

The monitor loop provides two levels of token protection:

12.3.1 Soft Limit (token_budget)

If total token usage across all changes exceeds the `token_budget` value:

- Running loops can finish their current iteration
- New changes **do not** start
- As soon as usage drops below the budget, dispatch resumes

12.3.2 Hard Limit (token_hard_limit)

If total tokens (including previous replan cycles) reach the hard limit:

- **Checkpoint** activates: the orchestrator stops
- Human approval is required to continue
- After approval, the limit is raised

```
wt-orchestrate approve   # approve, continue
```

12.4 Time Limit

The `time_limit` directive (default: 5 hours) limits **active** runtime:

```
wt-orchestrate --time-limit 4h start     # 4 hour limit
wt-orchestrate --time-limit none start   # no limit
```

When the limit expires:

1. Status changes to `time_limit`
2. Notification sent
3. Summary email sent
4. HTML report generated
5. Execution stops, but worktrees remain

Execution can be resumed: `wt-orchestrate start` — the timer continues where it left off.

12.5 Memory and Auditing

Every 10 polls (~2.5 minutes), the monitor loop automatically:

- `orch_memory_stats()`: memory system health
- `orch_gate_stats()`: gate statistics
- `orch_memory_audit()`: memory audit (duplicate detection)

This ensures the memory system remains healthy during long runs.

Chapter 13

Quality Gates

13.1 Overview

When a Ralph loop finishes its work (`loop-state.status == "done"`), the change is not immediately merged. It must pass through a sequential quality pipeline where every "gate" checks a different aspect of the work.

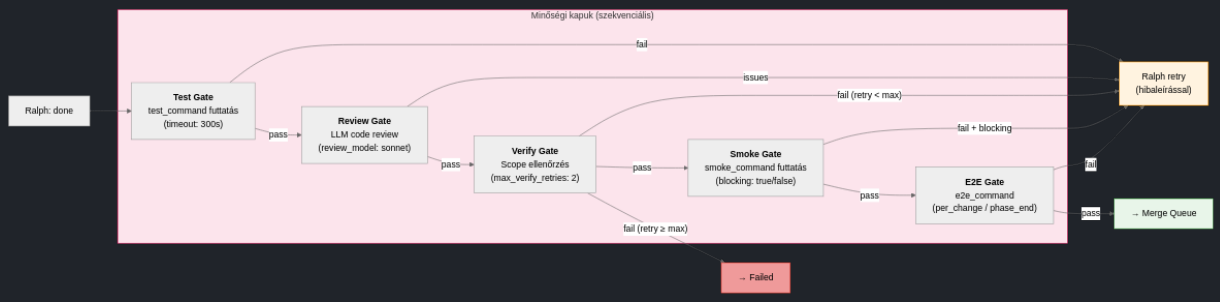

Figure 13.1: The sequential quality gates pipeline

13.2 Test Gate

The first and most important gate. If a `test_command` is configured, the system runs tests in the worktree.

13.2.1 How It Works

```
# The verifier.sh run_tests_in_worktree() function:
cd <worktree_path>
timeout <test_timeout> bash -c "<test_command>"
```

13.2.2 Parameters

Parameter	Default	Description
test_command	"" (skip)	The test command to run

Parameter	Default	Description
test_timeout	300s	Timeout for test execution

13.2.3 Output Handling

- **Pass** (exit code 0): Proceed to the next gate
- **Fail** (non-zero exit code): The test output (max 2000 characters) is saved, and the change goes back to the Ralph loop with the error message as retry context

Test statistics (`test_stats`) are saved to the state file:

```
{
  "test_result": "pass",
  "test_stats": {
    "exit_code": 0,
    "duration_ms": 12500,
    "output_tail": "Tests: 42 passed, 42 total"
  }
}
```

13.3 Review Gate

LLM-based code review, performed by a (typically) smaller and faster model.

13.3.1 How It Works

The system:

1. Collects the modifications made in the change (`git diff`)
2. Adds the change scope and assigned requirements
3. Sends it to the `review_model` (default: sonnet)
4. The LLM evaluates code quality

13.3.2 Requirement-Aware Review

`build_req_review_section()` appends the relevant requirements to the review prompt:

```
## Assigned Requirements (this change owns these)
- REQ-001: JWT authentication — Token-based auth on /api/* endpoints
- REQ-004: Token refresh — Automatic refresh of expired tokens

## Cross-Cutting Requirements (awareness only)
- REQ-012: Logging — Log all API calls
```

This helps the LLM evaluate not just code quality, but **requirement compliance** as well.

13.3.3 Configuration

Parameter	Default	Description
review_before_merge	false	Enable review gate
review_model	sonnet	Model used for review
skip_review	false	Change-level skip (in plan)

13.4 Verify Gate

The verify gate checks whether the implementation matches the scope.

13.4.1 How It Works

The `verify_merge_scope()` function:

1. Compares the planned scope with actual modifications
2. Checks that essential changes were made
3. Identifies unwanted side effects (scope creep)

13.4.2 Retry Logic

If the verify gate fails:

```
verify_retry_count < max_verify_retries?
  → yes: retry_context + Ralph loop restart
  → no: change → failed
```

The retry context includes the verify error message, build output, and original scope, so the agent can fix the problem in a targeted way.

Parameter	Default	Description
max_verify_retries	2	Maximum retry count

13.5 Smoke Gate

The smoke test is a lightweight, fast check that verifies build integrity.

13.5.1 How It Works

```
cd <worktree_path>
timeout <smoke_timeout> bash -c "<smoke_command>"
```

13.5.2 Blocking vs Non-blocking

Mode	Behavior
smoke_blocking: true	Failed smoke → Ralph retry (fix cycle)
smoke_blocking: false	Failed smoke → warning, but merge allowed

13.5.3 Smoke Fix Cycle

If the smoke test fails and `smoke_blocking: true`:

1. The error description is passed as retry context
2. Ralph loop starts for the smoke fix (limited budget)
3. Max `smoke_fix_max_retries` (default: 3) attempts
4. If unsuccessful → change failed

Parameter	Default	Description
smoke_command	""	Smoke test command
smoke_timeout	120s	Smoke timeout
smoke_blocking	true	Blocking mode
smoke_fix_token_budget	500K	Fix budget
smoke_fix_max_turns	15	Max fix iterations
smoke_fix_max_retries	3	Max retries

13.5.4 Health Check

Optionally, an HTTP health check can be configured:

```
smoke_health_check_url: "http://localhost:3000/api/health"
smoke_health_check_timeout: 30
```

After the smoke test, the system checks whether the application responds to HTTP.

13.6 E2E Gate

The most comprehensive test, examining the full application.

13.6.1 Two Modes

Mode	Description
e2e_mode: per_change	E2E runs before every change merge
e2e_mode: phase_end	E2E runs only at phase end, on the main branch

The `phase_end` mode is more efficient for large projects, because E2E tests are often slow and produce more relevant results when run on the main branch.

Parameter	Default	Description
e2e_command	""	E2E test command
e2e_timeout	120s	E2E timeout
e2e_mode	per_change	per_change/phase_end

13.7 Gate Order Summary

```
Ralph done
  → Test Gate (test_command, if configured)
    → Review Gate (review_before_merge, if active)
      → Verify Gate (scope check)
        → Smoke Gate (smoke_command, if configured)
          → E2E Gate (e2e_command, if configured)
            → ✓ Merge Queue
```

KEY

Every gate is optional. If there is no test_command, the Test Gate is skipped. If review_before_merge is false, the Review Gate is skipped. With minimal configuration (no gates at all), the change goes directly to the merge queue after Ralph is done — but this is not recommended for production projects.

13.8 Hooks

Hooks can run between gates:

Hook	When It Runs
hook_post_verify	After verify gate passes
hook_pre_merge	Before merge (blocking)
hook_on_fail	When a change enters failed status

Chapter 14

Merge and Delivery

14.1 Merge Queue

When a change successfully passes all quality gates, it enters the merge queue. The queue is processed according to the `merge_policy` directive.

14.2 Merge Policy

Three merge policies exist:

14.2.1 eager

The change is merged immediately as soon as it reaches the end of the verify pipeline.

```
merge_policy: eager
```

Advantage: fast delivery, other worktrees get fresh main sooner. Disadvantage: no opportunity for batch review.

14.2.2 checkpoint (default)

After N changes, the system stops and requests human approval.

```
merge_policy: checkpoint
checkpoint_every: 3
```

Process:

1. The 3rd change completes → checkpoint activates
2. The orchestrator stops (status: `checkpoint`)
3. The human reviews the merge queue
4. `wt-orchestrate approve --merge` → merge + continue

14.2.3 manual

Every merge requires human approval.

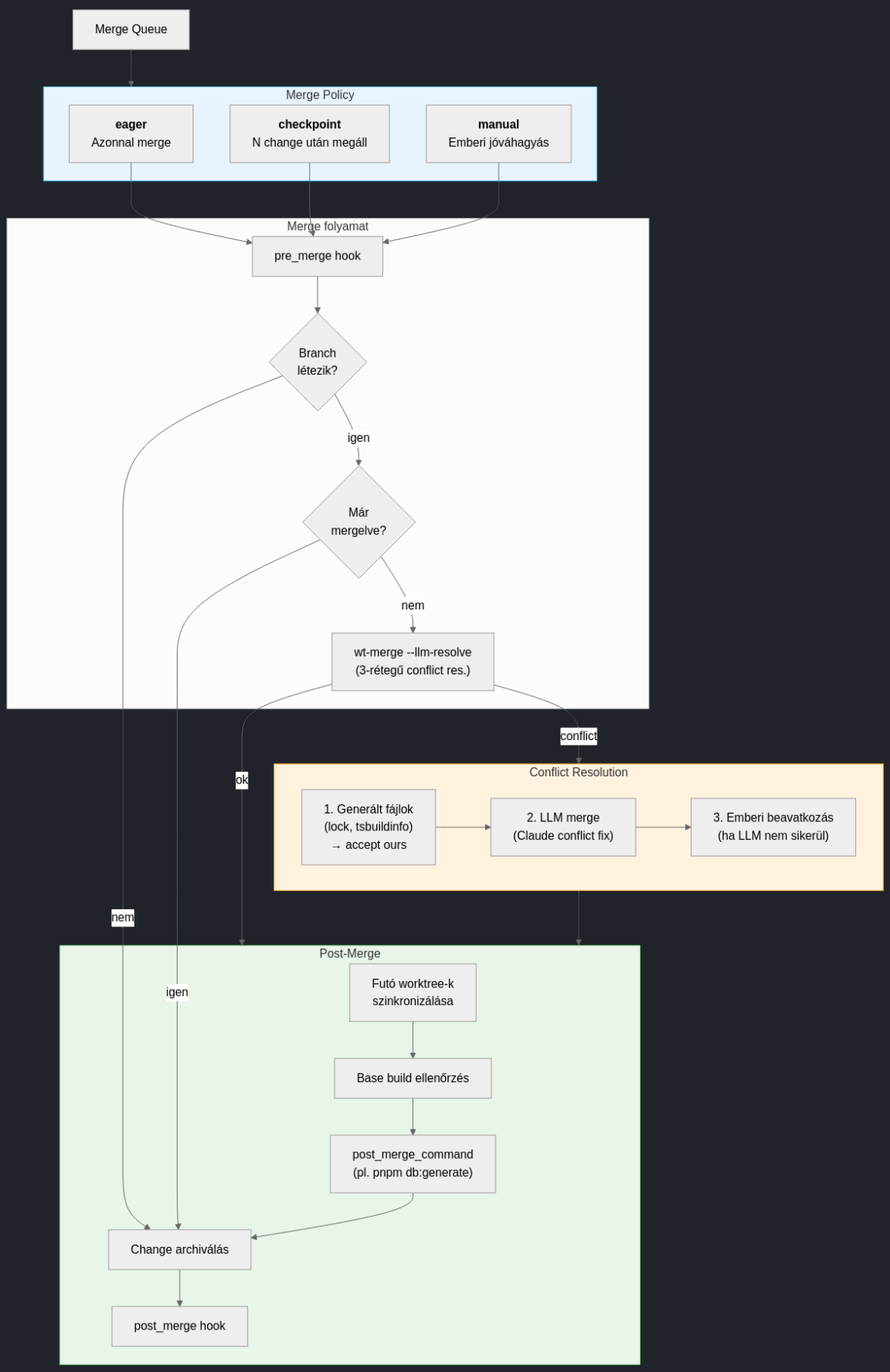

Figure 14.1: The merge pipeline and post-merge flow

```
merge_policy: manual
```

14.3 The Merge Process

`merge_change()` handles three cases:

14.3.1 Case 1: Branch No Longer Exists

If the `change/<name>` branch no longer exists (someone manually merged and deleted it):

- Status → `merged`
- Worktree cleanup
- Change archival

14.3.2 Case 2: Branch Already Merged

If the branch exists but is already an ancestor of HEAD:

- Status → `merged`
- Cleanup

14.3.3 Case 3: Normal Merge (the common case)

The `wt-merge` command performs the actual merge:

```
wt-merge <change-name> --no-push --llm-resolve
```

14.4 3-Layer Conflict Resolution

In case of merge conflicts, the system attempts resolution at three levels:

14.4.1 Layer 1: Generated Files (automatic)

`sync_worktree_with_main()` and the merge logic automatically handle generated file conflicts:

- `package-lock.json`
- `yarn.lock`, `pnpm-lock.yaml`
- `*.tsbuildinfo`

These are resolved with `--ours` strategy (the worktree version wins).

14.4.2 Layer 2: LLM Merge (--llm-resolve)

If there is a real code conflict, `wt-merge --llm-resolve` calls a Claude agent:

1. The conflicted files are passed to the LLM
2. The LLM understands the intent of both sides
3. The LLM merges the changes
4. The result is committed

14.4.3 Layer 3: Human Intervention

If the LLM cannot resolve the conflict either:

- The change moves to `merge_blocked` status
- Notification sent
- The human performs a manual merge
- The orchestrator detects the new state on the next poll

14.5 Post-Merge Pipeline

After a successful merge, the following steps run:

14.5.1 1. Running Worktree Synchronization

`_sync_running_worktrees()` ensures that other active worktrees receive the fresh main:

```
main ← merge(auth-system)
  ↓ sync
  worktree/user-profile ← git merge main
  worktree/api-endpoints ← git merge main
```

This prevents other agents from building on stale code.

14.5.2 2. Base Build Check

After merge, the base build cache is invalidated (`BASE_BUILD_STATUS=""`), and the next verify gate rechecks that the main branch is buildable.

14.5.3 3. Post-Merge Command

If a `post_merge_command` is configured:

```yaml
post_merge_command: "pnpm db:generate"
```

This command runs in the project root after merge. Typical uses:

- Database migration generation (`prisma generate`, `drizzle-kit generate`)
- Build artifact refresh
- Cache invalidation

14.5.4 4. Change Archival

`archive_change()` moves the OpenSpec change directory to the archive:

```
openspec/changes/auth-system/
  → openspec/changes/archive/2026-03-10-auth-system/
```

14.5.5 5. Post-Merge Hook

If a `hook_post_merge` is configured, it runs with the change name as argument.

14.5.6 6. Coverage Update

`update_coverage_status()` updates requirement coverage: the REQ-XXX identifiers assigned to the change move to `merged` status.

14.6 Checkpoint and Approval

14.6.1 Checkpoint Activation

Checkpoints activate in the following cases:

Trigger	Description
`checkpoint_every: N`	After N merges
`token_hard_limit`	Token hard limit reached
Manual	`wt-orchestrate pause --all`

14.6.2 Approval Process

```
wt-orchestrate approve           # approve, continue
wt-orchestrate approve --merge   # approve + immediate merge flush
```

The `--merge` flag immediately merges changes waiting in the merge queue.

14.6.3 Auto-approve (for E2E runs)

```
checkpoint_auto_approve: true
```

This is used for unsupervised runs (e.g., CI/CD, E2E testing).

> **KEY**
>
> The merge policy should be tuned to the project and team needs. For small projects, the eager policy is most efficient. For larger projects, the checkpoint policy ensures humans see the results before too many changes are merged.

14.7 Merge Retry Intelligence

If a merge fails (e.g., build error after merge), the system:

1. Saves the build output (`build_output`)
2. Moves the change to `merge_blocked` status
3. Retries on the next poll
4. If `fix_base_build_with_llm()` is available, attempts to fix the error with LLM

Chapter 15

Lessons Learned and Challenges

15.1 Solved Problems

The following were discovered during live orchestration runs — typically lessons from the first few production runs (sales-raketa, MiniShop, CraftBrew).

15.1.1 Token Budget Calibration

Problem: The original budget tiers (S=500K, M=2M) were too low. Artifact generation overhead (proposal → design → specs → tasks) already requires 500K-800K tokens before implementation even starts.

Solution: Calibration from live data: S=2M, M=5M, L=10M. Production runs confirmed: a typical S change uses 1.3-1.5M tokens (including artifact overhead).

15.1.2 Watchdog Spam

Problem: The watchdog emitted `WATCHDOG_WARN` events every 15 seconds — for a slow change, 60+ entries in the event log, degrading sentinel liveness detection and bloating log size.

Solution: Throttle event emission: only write to the log every 20th occurrence.

15.1.3 Sentinel Filename Mismatch

Problem: The sentinel watched `orchestration-events.jsonl` (hardcoded), but the orchestrator wrote to `orchestration-state-events.jsonl` (dynamic name). Result: the sentinel killed healthy orchestrators after 180 seconds.

Solution: The sentinel now derives the events filename from the state filename instead of hard-coding.

15.1.4 Misleading Token Tracking

Problem: Cache tokens counted toward the budget but didn't appear in tracking — 18x discrepancy between displayed and actual consumption.

Solution: Include cache tokens in tracking.

15.1.5 Large Change Reliability

Problem: 14+ requirements in a single change → 40+ minutes, 3 verify retries → failed. The cause: the context window isn't large enough, the agent loses track.

Solution: The planner applies a max 6 REQ/change rule. Changes with 4-6 requirements run reliably (12-19 minutes, 0 retries).

15.1.6 Jest + Playwright Collision

Problem: Jest unit tests crashed on Playwright `.spec.ts` files ("TypeError: Class extends value undefined") because jsdom can't handle browser imports.

Solution: `testPathIgnorePatterns` in jest config + per-worktree port isolation for Playwright tests.

15.2 Known Limitations

15.2.1 Mock-Based Tests Hide Runtime Errors

Problem: Jest tests mock Next.js APIs (`cookies()`, `headers()`), so they don't catch runtime errors. In the MiniShop run, 81 Jest tests were green while the application had 3 critical runtime bugs (auth bypass, cookie crash, dead link).

Lesson: Smoke testing (`pnpm build && pnpm test`) and E2E testing (`Playwright`) are needed together. Build catches type errors, E2E catches functional regressions.

15.2.2 Merge Conflicts on Shared Resources

Problem: Parallel changes modify the same file (e.g., `functional-conventions.md`, layout components). LLM merge can't resolve 900+ line conflicts.

Future: The planner should recognize cross-cutting files and serialize changes instead of parallelizing them.

15.2.3 Context Loss After Merge

Problem: After a schema change merge: code compiled correctly in the worktree, but on main (where other changes also modified the schema) TypeScript errors appeared.

Lesson: Post-merge build verification (`base build health check`) is essential.

15.2.4 Memory Doesn't Guarantee Quality

Problem: Benchmark runs showed that agents with memory explored 50% less but produced lower quality code — memory provides "shortcuts" that bypass the thorough code reading needed for complex changes.

Lesson: "Recall-then-verify" pattern: after memory recall, always verify the current state of the codebase.

15.3 What's Still Ahead

15.3.1 Cascade Failure Logic

When a change fails, dependent changes currently remain pending forever. Needed: automatic cascade failure propagation that moves dependent changes to `skipped` status.

15.3.2 Trend-Based Token Budget

Static budget limits (S=2M, M=5M) don’t scale to every project. Needed: project-level learning from actual token consumption, automatic limit adjustment.

15.3.3 Agent Scoring for Dispatch

Currently dispatch is round-robin. Needed: scoring agents by error rate, token efficiency, and iteration/progress ratio — better agents get more tasks.

15.3.4 Circuit Breaker for API Calls

On consecutive API errors, the system doesn’t stop but keeps trying. Needed: circuit breaker pattern that stops after N consecutive failures.

KEY

The real value of orchestration shows from the second run onward. The first run always reveals problems — wrong budget, missing test configuration, unexpected merge conflicts. The point: every error occurs only once, because the system learns from it (watchdog tuning, budget calibration, planner rules).

Chapter 16

Replan and Coverage

16.1 Auto-Replan

Auto-replan handles multi-phase specifications. When all changes in a phase are completed, the system automatically plans and starts the next phase.

16.1.1 Activation

```
auto_replan: true
```

16.1.2 Process

1. The monitor loop detects that every change is `merged` or `failed`
2. If `auto_replan: true` → `cmd_replan()` call
3. The planner receives the updated specification + the list of completed phases
4. A new plan is generated for the next phase
5. Previous token counters are saved (`prev_total_tokens`)
6. State is reset with the new plan
7. The monitor loop continues execution

16.1.3 Retry Logic

If replan fails (e.g., the LLM can't generate a good plan):

- Max `MAX_REPLAN_RETRIES` (3) consecutive attempts
- Failed replan → warning + wait for next poll
- If it fails 3 times → the orchestrator stops

16.1.4 All-Done Detection

The orchestrator considers the run "done" when:

1. There are no `pending` or `running` changes
2. There are no merge queue items
3. `auto_replan` is false OR the replan signals "no more phases"

```
Phase 1: [A, B, C] → merged → replan
Phase 2: [D, E]    → merged → replan
Phase 3: no more   → done
```

16.2 Manual Replan

The user can request a replan at any time:

```
wt-orchestrate replan
```

This is useful when:

- The specification changed during execution
- The user added new tasks
- Some changes failed and a new approach is needed

Manual replan:

1. Preserves the state of `merged` changes
2. Takes `failed` changes into account
3. Generates changes only for new/modified tasks

16.3 Requirement Coverage

The coverage system is based on the REQ-XXX identifiers from the digest. Its purpose: ensure that every requirement in the specification is covered by implementation.

16.3.1 Coverage Report

```
wt-orchestrate coverage
```

Output:

```
── Requirement Coverage ──

Coverage: 12/15 (80.0%)

Status breakdown:
  ✓ merged:     8
  ◐ running:    3
  ○ pending:    1
  ✗ failed:     1
  · unassigned: 2

Details:
  REQ-001  ✓  JWT authentication      → auth-system (merged)
  REQ-002  ✓  User profile            → user-profile (merged)
  REQ-003  ◐  Avatar upload           → user-profile (running)
  REQ-004  ✓  Token refresh           → auth-system (merged)
  ...
```

```
  REQ-014  ·  Admin audit log         → (not assigned)
  REQ-015  ·  Export function          → (not assigned)
```

16.3.2 Coverage Tracking Lifecycle

```
Plan generation:
  change.requirements = ["REQ-001", "REQ-004"]
  → populate_coverage() → coverage state init

During change execution:
  status = "running" → REQ-001 status = "running"

Change merge:
  update_coverage_status("auth-system", "merged")
  → REQ-001 status = "merged"
  → REQ-004 status = "merged"

Change failed:
  → REQ-XXX status = "failed"
  → check_coverage_gaps(): warning

Replan:
  Gaps (unassigned, failed) are included in the planner prompt
```

16.3.3 Coverage Gap Detection

`check_coverage_gaps()` watches for:

- **Unassigned**: No change is assigned to a REQ-XXX
- **Failed**: The change that owns the requirement has failed
- **Orphaned**: The requirement is in the digest but missing from the plan

These gaps are taken into account during replan, generating new changes to cover them.

16.3.4 Cross-Cutting Requirements

Some requirements may affect multiple changes. The `also_affects_reqs` field signals this:

```
{
  "name": "api-endpoints",
  "requirements": ["REQ-005"],
  "also_affects_reqs": ["REQ-012"]
}
```

REQ-012 (e.g., “Logging”) is not the primary task of `api-endpoints`, but it’s affected. The review gate takes this into account during evaluation.

16.4 Final Coverage Check

At the end of a run (done, time-limit), `final_coverage_check()` generates a summary report:

```
Final Coverage: 14/15 (93.3%)
  Uncovered: REQ-015 (Export function)
```

This summary is included in:

- The summary email
- The HTML report
- The orchestration-summary.md file

KEY

Coverage tracking is not a guarantee of correct implementation — it only indicates that a change "attempted" to cover the requirement. Actual quality is ensured by the verify and review gates. The main value of the coverage system: replan doesn't forget anything from the specification.

16.5 Summary Reports

16.5.1 HTML Report

`generate_report()` (in `reporter.sh`) generates a detailed HTML report:

- Per-change summary (status, token usage, time)
- Coverage matrix
- Gate statistics (pass/fail ratios)
- Timeline visualization

16.5.2 Summary Email

If email notification is configured, a summary email is sent at the end of a run:

- Runtime (active and wall)
- Change results
- Coverage summary
- Next steps (if replan is active)

16.5.3 orchestration-summary.md

A human-readable markdown summary generated at the end of a run.

Chapter 17

Reference

17.1 CLI Reference

17.1.1 wt-orchestrate

The main orchestration command.

```
wt-orchestrate [global options] <command> [command options]
```

17.1.1.1 Global Options

Option	Description
--spec <path>	Specification file or directory
--brief <path>	Brief file (legacy)
--phase <hint>	Phase filter (with --spec)
--max-parallel <N>	Max parallel override
--time-limit <dur>	Time limit (e.g., ``4h'', ``2h30m'', ``none'')

17.1.1.2 Commands

Command	Description
plan [--show]	Generate plan or display existing
start	Start plan execution
status	Display current status
pause <name|--all>	Suspend a change or all
resume <name|--all>	Resume a change or all
replan	Replan from updated spec
approve [--merge]	Checkpoint approval
digest --spec <path>	Generate structured digest
coverage	Requirement coverage report
specs [list|show|archive]	Spec document management
events [opts]	Event log query

Command	Description
tui	Start terminal dashboard
self-test	Run internal tests

17.1.2 wt-new / wt-merge / wt-loop

Command	Description
wt-new <name>	Create new worktree and branch
wt-merge <name> [--llm-resolve]	Merge worktree to main
wt-loop start [opts]	Start Ralph loop
wt-loop status	Query loop status

17.2 Change State Machine

Changes move through the following states during their lifecycle:

Figure 17.1: The complete change state machine

17.2.1 States

State	Description
pending	Waiting for dispatch (dependencies not met)
dispatched	Worktree created, Ralph loop starting
running	Ralph loop active, development in progress
verifying	Verify pipeline running (test → review → verify → smoke)
verify-failed	Verify gate failed, retry pending
stalled	Watchdog detected the agent is stuck
paused	Manually suspended
waiting:budget	Waiting due to token budget limit
budget_exceeded	Token budget exceeded
merge_queue	Verify successful, waiting for merge
merging	Merge in progress
merge_blocked	Merge conflict, could not resolve
merged	Successfully merged to main branch
smoking	Post-merge smoke test running
smoke_failed	Smoke test failed
smoke_blocked	Smoke fix retry limit reached
completed	All gates successfully passed
failed	Permanently failed (retry limit reached)
skipped	Skipped (dependency failed)

17.2.2 Key State Transitions

```
pending → dispatched → running → verifying
  ↑                      ↕          ↓
  │                  stalled    merge_queue → merging → merged → completed
  │                      ↓          ↑
  │                  failed    verify-failed (→ running retry)
  │                                ↓
  └── skipped (dependency failed)  failed (retry exhausted)
```

17.3 Directive Table

Directives control system behavior. The tables below show all available settings by category. Most projects start with default values and only set the test commands (`test_command`, `smoke_command`) and parallelism level (`max_parallel`). The remaining directives are for fine-tuning, which is typically needed only after the first few runs.

17.3.1 Execution

These directives define the basic parameters of execution: how many agents work simultaneously, how merging happens, and how long the system can run.

Directive	Type	Default	Description
max_parallel	number	3	Max parallel worktrees
merge_policy	enum	checkpoint	eager / checkpoint / manual
checkpoint_every	number	3	Checkpoint merge count
time_limit	string	5h	Active time limit
pause_on_exit	bool	false	Pause when orchestrator stops
context_pruning	bool	true	Context window optimization
model_routing	string	off	Model routing strategy

17.3.2 Models

There are three model tiers: opus for heavy work (implementation), sonnet for faster tasks (review, simple changes), haiku for summarization. Model routing can automatically choose based on change complexity.

Directive	Type	Default	Description
default_model	string	opus	Implementation model
review_model	string	sonnet	Review model
summarize_model	string	haiku	Summarization model

17.3.3 Testing

Testing directives configure the quality gates. `test_command` is the most important: if empty, the test gate is skipped entirely. Smoke and E2E tests are optional but strongly recommended for production runs — smoke catches build errors, E2E catches functional regressions.

Directive	Type	Default	Description
test_command	string	""	Test command
test_timeout	number	300	Test timeout (seconds)
smoke_command	string	""	Smoke test command
smoke_timeout	number	120	Smoke timeout (seconds)
smoke_blocking	bool	true	Does smoke fail block
smoke_fix_token_budget	number	500K	Smoke fix token limit
smoke_fix_max_turns	number	15	Smoke fix max iterations
smoke_fix_max_retries	number	3	Smoke fix max retries
smoke_health_check_url	string	""	Health check URL
smoke_health_check_timeout	number	30	Health check timeout
e2e_command	string	""	E2E test command
e2e_timeout	number	120	E2E timeout (seconds)
e2e_mode	enum	per_change	per_change / phase_end

17.3.4 Review and Verify

Directive	Type	Default	Description
review_before_merge	bool	false	LLM review gate
max_verify_retries	number	2	Verify retry limit

17.3.5 Token Control

Token control manages costs. The soft limit (`token_budget`) warns and slows down dispatch, but running agents can finish their work. The hard limit (`token_hard_limit`) stops the system and requests human approval — this is the ultimate safety net against unexpected token consumption.

Directive	Type	Default	Description
token_budget	number	0	Soft limit (0=off)
token_hard_limit	number	20M	Hard limit
checkpoint_auto_approve	bool	false	Auto checkpoint approve

17.3.6 Planning

Directive	Type	Default	Description
auto_replan	bool	false	Auto-replan at phase end
plan_method	enum	api	api / agent
plan_token_budget	number	500K	Agent plan budget

17.3.7 Watchdog

The watchdog is the “night guard”: it intervenes when an agent is stuck and cannot proceed on its own. The timeout value should be tuned to the project’s build time — if builds take 5 minutes, the 600s timeout is correct; if they take 30 seconds, consider lowering it to 180s.

Directive	Type	Default	Description
watchdog_timeout	number	600	Stall timeout (seconds)
watchdog_loop_threshold	number	5	Loop detection threshold
max_redispatch	number	2	Max redispatch attempts

17.3.8 Hooks

Hooks allow project-specific logic to be injected into the pipeline. The `pre_merge` hook is blocking: if it throws an error, the merge does not happen. Other hooks are non-blocking — if they error, the pipeline continues. The `post_merge_command` is the most commonly used hook: typically for database migration generation (Prisma, Drizzle) or build artifact refresh.

Directive	Type	Default	Description
hook_pre_dispatch	string	""	Before dispatch
hook_post_verify	string	""	After verify

Directive	Type	Default	Description
hook_pre_merge	string	""	Before merge (blocking)
hook_post_merge	string	""	After merge
hook_on_fail	string	""	On failure
post_merge_command	string	""	Post-merge command

17.3.9 Event Log

Directive	Type	Default	Description
events_log	bool	true	Event log active
events_max_size	number	1MB	Max log size

17.4 File Structure Reference

17.4.1 Project-Level Files

```
.claude/
├── orchestration.yaml          ← configuration
└── orchestration.log           ← runtime log

wt/orchestration/
├── digest/
│   ├── requirements.json       ← REQ-XXX identifiers
│   ├── phases.json             ← phase structure
│   ├── digest-meta.json        ← hash, date
│   └── ambiguities.json        ← ambiguous points
└── specs/
    ├── v12.md                  ← active specs
    └── archive/
        └── v11.md              ← archived specs

orchestration-plan.json         ← current plan (gitignore)
orchestration-state.json        ← runtime state (gitignore)
orchestration-summary.md        ← summary (gitignore)
```

17.4.2 Worktree-Level Files

```
.claude/worktrees/<change-name>/
├── .claude/
│   └── loop-state.json         ← Ralph loop state
├── openspec/changes/<name>/
│   ├── proposal.md             ← OpenSpec proposal
│   ├── design.md               ← design document
│   └── tasks.md                ← task list
```

```
└── ... (project files)
```

17.4.3 Event Types

Event	Description
STATE_CHANGE	Change status transition
DISPATCH	Change dispatch
MERGE_ATTEMPT	Merge attempt
MERGE_SUCCESS	Successful merge
MERGE_FAIL	Failed merge
VERIFY_PASS	Verify gate pass
VERIFY_FAIL	Verify gate fail
TEST_PASS / TEST_FAIL	Test result
SMOKE_PASS / SMOKE_FAIL	Smoke test result
WATCHDOG_ESCALATE	Watchdog escalation
CHECKPOINT	Checkpoint activation
REPLAN	Replan start
DIGEST_STARTED / DIGEST_FAILED	Digest events

Events are written to the `.claude/orchestration.log` JSONL file and can be queried with the `wt-orchestrate events` command:

```
wt-orchestrate events --type MERGE_SUCCESS --last 10 --json
```